UNIVERZA NA PRIMORSKEM
FAKULTETA ZA MATEMATIKO, NARAVOSLOVJE IN INFORMACIJSKE TEHNOLOGIJE

ZAKLJUČNA NALOGA
(FINAL PROJECT PAPER)

PREDIKTIVNO VZDRŽEVANJE PROGRAMSKE OPREME Z UPORABO STROJNEGA UČENJA: PRIMERJALNA ŠTUDIJA ALGORITMOV NA PODATKOVNEM NABORU KC1 NASA PROMISE (PREDICTIVE MAINTENANCE FOR SOFTWARE SYSTEMS USING MACHINE LEARNING: A COMPARATIVE STUDY OF ALGORITHMS ON THE KC1 NASA PROMISE DATASET)

TEODORA TALESKA

UNIVERZA NA PRIMORSKEM
FAKULTETA ZA MATEMATIKO, NARAVOSLOVJE IN INFORMACIJSKE TEHNOLOGIJE

Zaključna naloga
(Final project paper)

Prediktivno vzdrževanje programske opreme z uporabo strojnega učenja: Primerjalna študija algoritmov na podatkovnem naboru KC1 NASA PROMISE

(Predictive Maintenance for Software Systems using Machine Learning: A Comparative Study of Algorithms on the KC1 NASA PROMISE Dataset)

Ime in priimek: Teodora Taleska
Študijski program: Računalništvo in informatika
Mentor: doc. dr. Domen Šoberl

Koper, 2024

This work is offered under the *Creative Commons Attribution-ShareAlike* license version 2.5 Slovenia (or newer). This means that both the text, images, graphics, and other components of the work, as well as the results of the thesis, can be freely distributed, reproduced, used, made available to the public, and modified, provided that the author and title of this work are clearly and visibly attributed. In the case of any changes, transformations, or use of this work in another project, the derivative work can only be distributed under a license that is identical to this one. Detailed information about the license is available on the website `http://creativecommons.si/` or at the Institute for Intellectual Property, Streliška 1, 1000 Ljubljana.

The source code of the thesis, its results, and the software developed for this purpose are offered under the GNU General Public License, version 3 (or newer). This means that they can be freely distributed and/or modified under its terms. Detailed information about the license is available on the website `http://www.gnu.org/licenses/`.

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my mentor, doc. dr. Domen Šoberl, for his invaluable guidance, support, and encouragement throughout this project. His expertise and insights were crucial in the successful completion of this work. Thanks also to the University of Ottawa for their PROMISE (Predictor Models in Software Engineering) repository, providing us with the publicly available datasets that serve researchers in building predictive software models.

I am also grateful to my colleagues who provided helpful discussions and feedback. Their collaboration and suggestions were instrumental in refining this research.

Lastly, I would like to thank my family and friends for their continuous support and understanding throughout this project.

Ključna dokumentacijska informacija

Ime in PRIIMEK: Teodora TALESKA

Naslov zaključne naloge: Prediktivno vzdrževanje programske opreme z uporabo strojnega učenja: Primerjalna študija algoritmov na podatkovnem naboru KC1 NASA PROMISE

Kraj: Koper

Leto: 2024

Število strani: 47 Število slik: 8

Število tabel: 6 Število referenc: 22

Mentor: doc. dr. Domen Šoberl

Ključne besede: Napovedno vzdrževanje, Strojno učenje, KC1, NASA, PROMISE

Izvleček:
Ta diplomska naloga preiskuje učinkovitost strojnega učenja pri napovedovanju programskih napak, pri čemer uporablja značilke, izpeljane iz meril McCabe in Halstead. Raziskava uporablja tradicionalne algoritme strojnega učenja, vključno z umetnimi nevronskimi mrežami (ANN), regresijo podpornih vektorjev (SVR), linearno regresijo (LR) ter odločitvenimi drevesi (DT), skupaj z dvema metodama sestavljenih modelov, naključnim gozdom (RF) in gradientnim boostingom (GBR). Uspešnost teh modelov smo ocenili s pomočjo prečnega preverjanja ter z metodo izločitve enega primera.

Rezultati kažejo, da je nevronska mreža z dvema skritima nivojema najbolj učinkovita med uporabljenimi metodami, saj dosledno dosega najvišje točnosti glede na uporabljene metrike. To učinkovitost lahko pripišemo njeni sposobnosti modeliranja kompleksnih vzorcev v podatkih, kar vodi do zelo točnih napovedi. Nasprotno pa so se odločitvena drevesa izkazala kot najmanj učinkoviti model, saj so kazala najvišjo srednjo kvadratno napako (MSE) in korensko srednjo kvadratno napako (RMSE), še posebej pri vključitvi izstopajočih primerov, kar je vplivalo na njihovo napovedno točnost in posplošitev.

Naša raziskava poudarja prednosti naprednih tehnik strojnega učenja, zlasti nevronskih mrež in metod sestavljenih modelov, pri izboljšanju točnosti in robustnosti napovedovanja programskih napak. Te ugotovitve zagotavljajo dragoceno osnovo za nadaljnje prakse napovednega vzdrževanja programske opreme, poudarjajoč ključno vlogo sofisti-

ciranih metod strojnega učenja pri ohranjanju kakovosti in zanesljivosti programske opreme.

Key document information

Name and SURNAME: Teodora TALESKA

Title of the final project paper: Predictive Maintenance for Software Systems using Machine Learning: A Comparative Study of Algorithms on the KC1 NASA PROMISE Dataset

Place: Koper

Year: 2024

Number of pages: 47 Number of figures: 8

Number of tables: 6 Number of references: 22

Mentor: assist. prof. Domen Šoberl, PhD

Keywords: Predictive maintenance, Machine learning, KC1, NASA, PROMISE

Abstract:
This thesis investigates the effectiveness of machine learning in predicting software defects, utilizing features derived from McCabe and Halstead metrics. The study employs traditional machine learning algorithms, including Artificial Neural Networks (ANN), Support Vector Regressor (SVR), Linear Regression (LR), and Decision Trees (DT), alongside two ensemble methods, Random Forest (RF) and Gradient Boosting Regressor (GBR). The performance of these models was evaluated using leave-one-out cross-validation and 6-fold cross-validation.

The results demonstrate that an artificial neural network with two hidden layers is the most effective among the tried methods, consistently achieving the best performance across the used metrics. This effectiveness is attributed to its capability to model complex patterns in the data, leading to highly accurate predictions. In contrast, Decision Trees emerged as the least effective model, exhibiting the highest Mean Squared Error (MSE) and Root Mean Squared Error (RMSE), particularly with outliers included, which impaired their predictive accuracy and generalizability.

The study underscores the advantages of advanced machine learning techniques, particularly neural networks and ensemble methods, in enhancing prediction accuracy and robustness. These findings provide a valuable foundation for advancing predictive maintenance practices in software engineering, highlighting the critical role of sophisticated machine learning methods in maintaining software quality and reliability.

Contents

1 INTRODUCTION **1**
1.1 BACKGROUND AND MOTIVATION 1
1.2 OBJECTIVES OF THE STUDY 2
1.3 RESEARCH QUESTIONS 3
1.4 STRUCTURE OF THE THESIS 3
1.5 LITERATURE REVIEW 3
1.5.1 Software Architecture 3
1.5.2 Measuring Software Complexity 5
1.5.3 Overview of Predictive Maintenance 9
1.5.4 Machine Learning in Predictive Maintenance 9
1.5.5 Review of Related Work 10

2 METHODOLOGY **11**
2.1 DESCRIPTION OF THE NASA KC1 DATASET 11
2.2 EXPERIMENTAL SETUP 16
2.2.1 Data Preprocessing 16
2.2.2 Training and Testing Procedure 17
2.2.3 Hyperparameter Tuning 18
2.2.4 Feature Importance 19
2.3 THE BASELINE MODEL 20
2.4 MACHINE LEARNING ALGORITHMS USED 21
2.4.1 Artificial Neural Networks 21
2.4.2 Support Vector Regressor 25
2.4.3 Linear Regression 27
2.4.4 Decision Tree Regressor 28
2.5 ENSEMBLE METHODS 31
2.5.1 Random Forest 32
2.5.2 Gradient Boosting Regressor 33
2.6 Evaluation Metrics 35
2.6.1 Correlation Coefficient 35
2.6.2 Mean Absolute Error 36
2.6.3 Mean Squared Error 36
2.6.4 Root Mean Squared Error 36

3 RESULTS AND DISCUSSION **37**

3.1 ANALYSIS OF OUTLIERS 37
3.2 PERFORMANCE OF THE ALGORITHMS 38
3.3 COMPARATIVE ANALYSIS 39
3.3.1 Interpretation of Results 39
3.3.2 Comparison with Baseline 41
3.3.3 Best Performing Algorithm 41
3.3.4 Worst Performing Algorithm 42

4 CONCLUSION 45

5 DALJŠI POVZETEK V SLOVENSKEM JEZIKU 46

6 BIBLIOGRAPHY 48

List of Tables

1 Attributes and their value ranges 13
2 Top 15 most important features 19
3 Model performance using leave-one-out cross-validation with outliers included 39
4 Model performance using leave-one-out cross-validation with outliers excluded 39
5 Model performance using 6-fold cross-validation with outliers included 40
6 Model performance using 6-fold cross-validation with outliers excluded 40

List of Figures

1 The fundamental architecture of a feed-forward network [1]. 23
2 Pre-activation and post-activation values within a neuron [1]. 23
3 One-dimensional linear SVR [3]. 26
4 Common ensemble architecture. 31
5 Predicted vs True Values (6-Fold CV) for ANN with hidden layers (200, 100). 42
6 Residuals Histogram for ANN with hidden layers (200, 100). 43
7 Predicted vs True Values (6-Fold CV) for Decision Tree Regressor. . . 44
8 Residuals Histogram for Decision Tree Regressor. 44

List of Abbreviations

PROMISE	Predictor Models in Software Engineering
RSS	Residual Sum of Squares
GBR	Gradient Boosting Regressor
SVR	Support Vector Regressor
LR	Linear Regression
ANN	Artificial Neural Network
DT	Decision Tree
RF	Random Forest
ML	Machine Learning
MDI	Mean Decrease Impurity
ReLU	Rectified Linear Unit

1 INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Predictive maintenance is an essential aspect of modern industrial systems, aimed at predicting equipment failures before they occur to minimize downtime and maintenance costs. With the increasing amount of data, generated by industrial systems, machine learning techniques have become pivotal in enhancing the accuracy and efficiency of predictive maintenance. This thesis explores the application of machine learning to improve predictive maintenance practices specifically in software systems.

The motivation behind this research stems from the high costs and operational challenges associated with traditional maintenance strategies. Any modifications made to a system after it has become operational are regarded as maintenance. Maintenance activities can be categorized into four main types: corrective, adaptive, perfective, and preventive [17]. Corrective maintenance focuses on fixing software problems after they occur, typically resulting in substantial downtime and costs. Adaptive maintenance entails modifying a particular part of the system to respond to changes in another part. Perfective maintenance involves making changes for the improvement of some aspect of the system, even if they were not a result of a fault. Preventive maintenance, on the other hand, involves performing maintenance activities at scheduled intervals, regardless of the actual condition of the system [17]. While preventive maintenance can reduce the likelihood of unexpected failures, it often leads to unnecessary maintenance actions, resulting in wasted resources and downtime when the software can still be operational.

Predictive maintenance offers a more efficient alternative by leveraging data to predict when maintenance should be performed, based on the actual condition of the system rather than a fixed schedule. This approach aims to optimize maintenance activities, reduce unnecessary interventions, and prevent unexpected failures, ultimately leading to significant cost savings and improved system reliability.

In the realm of software systems, maintenance is not only costly but also complex due to the inherent nature of software defects and the intricate interdependencies within the codebase. Software defects, if not identified and addressed on time, can lead to system failures, degraded performance, and increased operational costs. Traditional maintenance approaches may not be sufficient to address these challenges, as they often fail to predict when and where defects will occur.

The emergence of machine learning presents a promising solution to these challenges. Machine learning algorithms are capable of processing large datasets, detecting patterns, and making highly accurate predictions. By applying machine learning to software maintenance,

we can predict the likelihood of defects in software modules before they manifest into system failures. This enables us to perform maintenance activities in a more targeted and efficient manner, reducing downtime and maintenance costs.

The KC1 NASA PROMISE dataset [9] provides a valuable resource for this research. The dataset includes various metrics extracted from the source code, such as McCabe and Halstead metrics, which have been widely used to characterize software quality. These metrics provide a basis for training machine learning models to predict software defects. The dataset's availability and relevance to real-world software systems make it an ideal choice for exploring the effectiveness of different machine learning algorithms in predictive maintenance.

By predicting whether a software module is likely to be defective, we can plan maintenance activities more effectively. This proactive approach can prevent system failures, reduce downtime, and optimize resource utilization. Moreover, it allows for a more balanced maintenance strategy that avoids the pitfalls of preventive maintenance. The insights gained from this research will contribute to the field of predictive maintenance and offer practical implications for software development and maintenance practices.

1.2 OBJECTIVES OF THE STUDY

The primary objective of this study is to train and evaluate machine learning models for predictive maintenance using the KC1 NASA dataset from the PROMISE repository. The specific objectives are:

- To analyze the effectiveness of various machine learning algorithms in predicting maintenance needs.
- To explore the use of ensemble methods in enhancing prediction accuracy.

This study aims to research the feasibility of using machine learning methods on the KC1 dataset, which includes the McCabe and Halstead metrics extracted from source code, to predict software module defects. By exploring different algorithms and approaches, this study seeks to determine which models can most effectively predict the occurrence of software defects, enabling more efficient maintenance planning and execution.

By accurately predicting software defects, the study aims to contribute to the understanding of cost implications, system reliability, and maintenance optimization. This research provides a comprehensive analysis of various machine learning algorithms' effectiveness in predicting software defects and proposes enhancements to improve predictive accuracy.

1.3 RESEARCH QUESTIONS

The study aims to address the following research questions:

1. How well can machine learning help in predicting software defects based on the features derived from McCabe and Halstead metrics?
2. How much can ensemble machine learning models improve the accuracy of software defect prediction in predictive maintenance systems compared to traditional single-algorithm approaches?
3. Which are the key factors (features) that significantly impact the prediction of software defects in the KC1 dataset?

We will primarily focus on the first research question. Our methodology will include preprocessing the KC1 dataset, training various machine learning models, and evaluating their performance using regression evaluation metrics such as R^2, MAE, RMSE, and MSE. Additionally, we will explore the use of ensemble models to enhance prediction accuracy and compare their performance to traditional single-algorithm approaches.

By examining the key attributes that impact the prediction of software defects, we will provide insights into which features are most indicative of potential defects. Understanding these attributes will help us refine the models and further improve their prediction accuracy.

1.4 STRUCTURE OF THE THESIS

This thesis is structured as follows:

- *Chapter 1: Introduction* - Provides the background, objectives, research questions, and structure of the thesis, including the literature review.
- *Chapter 2: Methodology* - Describes the dataset, the used machine learning algorithms, and the experimental setup.
- *Chapter 3: Results and Discussion* - Presents the results of the experiments and discusses the findings in the context of the research questions.
- *Chapter 4: Conclusion* - Summarizes the main findings, discusses the implications of the study, and suggests directions for future research.

1.5 LITERATURE REVIEW

1.5.1 Software Architecture

The records of the dataset that we use are the results of McCabe and Halstead metrics that were applied to the modules of the KC1 NASA product. It is therefore essential to delve

into details about the software architecture. This section provides an overview of design principles, with a particular emphasis on the concept of modularity.

Modularity Principle

Modularity is a fundamental design principle in software engineering that involves dividing a software system into distinct modules that can be developed, tested and maintained independently [17]. Each module encapsulates a specific subset of the system's functionality and interacts with other modules through well-defined interfaces. Module independence makes it easier to locate faults because there are fewer suspect modules per fault, and to change the system, because a change to one module affects relatively few other modules. There are two concepts used for measuring module independence: cohesion and coupling [17].

Cohesion

Cohesion refers to the degree to which the elements within a module belong together [17]. High cohesion within a module means that the responsibilities of the internal elements, such as functions, data, and internal modules, are closely related, making the module more understandable and maintainable. Cohesion is typically classified into several types (listed from high to low) as specified in [17]:

- *Informational Cohesion:* The elements are grouped based on their relationship to a specific data structure or object.
- *Functional Cohesion:* The elements of the module work together to perform a single well-defined task.
- *Temporal Cohesion:* The elements are related by their timing, meaning they are used at the same time during execution.
- *Procedural Cohesion:* The elements of the module follow a specific sequence of execution.
- *Communicational Cohesion:* The elements of the module operate on the same data.
- *Logical Cohesion:* The elements are grouped by the logic structure of the code.
- *Coincidental Cohesion:* The worst degree of cohesion. The elements in the module have little or no meaningful relationship to one another.

High cohesion is desirable as it ensures that a module's responsibilities are closely aligned and focused, which simplifies maintenance and enhances reusability.

Coupling

Coupling describes the extent to which modules are interdependent [17]. It measures how closely connected different modules are to each other. Lower coupling is generally preferred

because it reduces the impact of changes in one module on others, thereby enhancing the system's maintainability and flexibility. Coupling can be classified into various types according to the nature and intensity of the connections between modules, as detailed in [17]:

- *Content Coupling:* One module directly accesses or modifies the content of another module.
- *Common Coupling:* Multiple modules use the same global data.
- *Control Coupling:* One module controls the flow of another by passing information on what to do (e.g., passing a control flag).
- *Stamp Coupling:* Modules share a composite data structure, and each uses only a part of it.
- *Data Coupling:* Modules exchange data via parameters, where each parameter represents a simple, individual piece of data.

Reducing coupling while maintaining or increasing cohesion results in a modular system that is easier to develop, test, and maintain [17].

1.5.2 Measuring Software Complexity

Software Measurement

Measurement is the process of assigning numbers or symbols to real-world attributes or entities in a manner that describes them according to specific, well-defined rules [5]. Software measurement is a critical aspect of software engineering that involves the quantification of various attributes of software products and processes. This systematic approach enables developers, project managers, and stakeholders to gain insights into the quality, performance, and maintainability of software systems. The primary goal of software measurement is to provide objective data that can be used to make informed decisions throughout the software development life cycle.

Importance of Software Measurement

Software measurement is essential for several reasons:

- *Quality Assurance*: Measurement helps in assessing and ensuring the quality of software products. By measuring attributes such as code complexity, maintainability, and defect density, teams can identify potential issues early and implement corrective actions.
- *Project Management*: Accurate measurements enable better planning, estimation, and tracking of software projects. Metrics such as effort estimation, schedule adherence, and cost analysis are crucial for successful project management.

- *Process Improvement*: Continuous measurement allows organizations to identify areas for process improvement. By analyzing metrics over time, teams can refine their development processes to enhance efficiency and productivity.
- *Benchmarking and Comparison*: Metrics provide a basis for benchmarking and comparing different software systems, teams, and processes. This comparison helps in setting standards and striving for best practices.

Measuring Internal Product Attributes: Size and Structure

When it comes to measuring the internal attributes of software products, two fundamental aspects are considered: size and structure [5]. These attributes provide insights into the complexity and maintainability of the software.

Size

Each software development product is a tangible entity and can be characterized by its size. Size is one of the most apparent and valuable attributes of a software system. It can be assessed statically, without the need to run the system. Key aspects of size include *length* (the physical size), *functionality* (the functions provided to the user by the product), and *complexity* (the complexity of the problem the software is addressing) [5].

The three major development products whose size would be useful to know are the specification, the design, and the code. Code can be produced in several ways. The most conventional approach is to use a procedural programming language. The most commonly used measure for source code program length is the number of lines of code (LOC) [5]. However, not all lines of code are the same. As developers write code, they sometimes leave blank lines of code for more readable code, comment lines are added for the code to be more easily understood by other programmers in the team, data declarations, etc. Therefore, the lines of code can be divided into the following categories:

- blank lines,
- comment lines,
- data declarations,
- lines that contain several separate instructions.

The Hewlett-Packard definition of a line of code is the most widely accepted standard. According to Hewlett-Packard, a line of code is defined as a source statement that excludes comments and blank lines: any statement within the program that is not a comment or an empty line [5]. Thus, the total length of the source code, as measured by lines of code (LOC), can be defined as:

$$\text{total length (LOC)} = NCLOC + CLOC \tag{1.1}$$

where

- $NCLOC$: Non-commented line.
- $CLOC$: Commented lines.

Halstead's Approach

Maurice Halstead introduced a set of software metrics, known as Halstead's metrics, which quantify the size and complexity of a software program based on its source code. Halstead's approach includes the following key metrics [5]:

- n_1: The total number of distinct operators in the program.
- n_2: The total number of distinct operands in the program.
- N_1: The total number of times operators appear in the program.
- N_2: The total number of times operands appear in the program.

From these basic metrics, several derived metrics can be calculated:

$$\text{Program Length (N)} = N_1 + N_2 \tag{1.2}$$

$$\text{Program Vocabulary (n)} = n_1 + n_2 \tag{1.3}$$

The *volume* of the program, akin to the number of mental comparisons needed to write a program of length N, is

$$\text{Volume (V)} = N \times \log_2(n) \tag{1.4}$$

The *program level* of a program *P* of volume *V* is

$$\text{Program level (L)} = \frac{V*}{V} \tag{1.5}$$

where *V** is the potential volume, the volume of the minimal size of implementation of P. The inverse level is the *difficulty*:

$$\text{Difficulty (D)} = \frac{1}{L} \tag{1.6}$$

The *effort* required to generate *P* is given by

$$\text{Effort (E)} = \frac{n_1 \times N_2 \times N \times \log_2(n_2)}{2 \times n_2} \tag{1.7}$$

These metrics offer a quantitative assessment of the software's complexity and the effort needed for development and maintenance [5].

Structure

The structure as a product attribute can refer to the structure of requirements, design, and code. This structure may help us understand the difficulty of converting one product to another, for instance, implementing a design as code, testing a product, or predicting external software attributes from early internal product measures. Although the structural measures vary in what they measure and how they measure it, they are often called "complexity" metrics [5].

Structure or complexity encompasses various aspects, each serving a distinct function. At a minimum, the structure can be considered to consist of the following three components:

- control-flow structure — the sequence in which instructions are executed in a program.
- data-flow structure — follows the trail of a data item as it is created or handled by a program.
- data structure — the organization of the data itself, independent of the program.

McCabe's Cyclomatic Complexity Measure

Cyclomatic complexity measures the number of linearly independent paths through a program's source code. A higher cyclomatic complexity indicates a more complex codebase, which may be more challenging to maintain. Introduced by Thomas McCabe, cyclomatic complexity assesses the complexity of a program's control flow. It is derived from the program's control flow graph (CFG), where nodes represent blocks of code and edges denote the control flow between these blocks [5].

McCabe proposed that program complexity can be measured by the cyclomatic number of the program's flow graph [5]. The cyclomatic number measures the number of *linearly independent* paths through *F* and it is calculated using the following formula:

$$v(F) = e - n + 2 \tag{1.8}$$

where

- F is the flow graph,
- e is the number of edges in the control flow graph,
- n is the number of nodes in the control flow graph.

McCabe's Essential Complexity Measure

Essential complexity is another metric introduced by McCabe, which focuses on capturing the overall level of structuredness in a program [5]. For a program with flow graph *F*, the *essential complexity* (*ev*) is defined as:

$$ev(F) = v(F) - m \tag{1.9}$$

where

- m is the number of subflow graphs of F that are D-structured primes.

1.5.3 Overview of Predictive Maintenance

In everyday language, "prediction" often refers to forecasting future events. However, in the realm of data analytics, prediction involves assigning values to unknown variables [8]. This might include forecasting future product prices or estimating the number of defects in a component.

Predictive maintenance employs data analysis tools and techniques to identify anomalies in equipment performance and forecast potential failures. This proactive method enables timely maintenance scheduling and helps prevent unexpected breakdowns. Unlike traditional maintenance strategies, which are either reactive or based on average lifespan statistics, predictive maintenance focuses on the actual operating conditions of equipment to decide when maintenance is necessary. This method reduces unscheduled breakdowns, optimizes machinery availability, and lowers overall maintenance costs. Additionally, it enhances product quality, productivity, and the profitability of manufacturing and production processes [16].

1.5.4 Machine Learning in Predictive Maintenance

Supervised and unsupervised machine learning techniques have been extensively applied in predictive maintenance. These techniques analyze historical data to build models for predicting future equipment failures. Supervised learning involves training a model on labeled data, allowing it to learn the relationship between input features and the target variable, such as the likelihood of failure. Unsupervised learning, on the other hand, helps in identifying patterns and anomalies in the data without predefined labels, which can be crucial for early detection of potential issues [8].

The implementation of machine learning in predictive maintenance encompasses several crucial steps. First, data is collected from various sources, including sensors and historical maintenance records. The data is then preprocessed to handle issues such as missing values, outliers, and noise. Feature engineering is performed to extract relevant features that can help in predicting failures. Next, machine learning models are trained on this data to learn patterns and correlations. These models are then evaluated and fine-tuned to improve their accuracy and reliability. Finally, the trained models are deployed in a live environment to monitor equipment performance in real-time and provide early warnings of potential failures.

1.5.5 Review of Related Work

A variety of studies have investigated the use of machine learning algorithms for predicting software defects. Marwa Assim et al. [2] conducted a comprehensive investigation using various machine learning algorithms to predict software defects. The study implemented algorithms such as artificial neural networks (ANNs), random forest (RF), random tree (RT), decision table (DT), linear regression (LR), Gaussian processes (GP), SMOreg, and M5P, utilizing the Weka tool for evaluation. Their results indicated that the SMOreg classifier achieved the best performance, while the ANN classifier performed the worst.

Koru and Liu [10] conducted a detailed investigation into the effect of module size on defect prediction using static measures. The study utilized several machine learning algorithms to predict defects in five NASA software products (CM1, JM1, KC1, KC2, and PC1). The researchers used static measures such as lines of code (LOC) as predictor variables and discovered that prediction performance improved significantly for larger modules. Furthermore, for the KC1 software product, the study demonstrated that using class-level data for prediction, as opposed to method-level data, resulted in enhanced prediction performance. These findings suggest that quality assurance efforts can be more efficiently directed by focusing on larger modules.

2 METHODOLOGY

This study employs an empirical research design to evaluate the effectiveness of various machine learning algorithms in predictive maintenance using the KC1 NASA dataset from the PROMISE repository. The research design includes data preprocessing, model training, evaluation, and comparison of results.

2.1 DESCRIPTION OF THE NASA KC1 DATASET

The NASA KC1 dataset is a comprehensive collection of class-level metrics aimed at facilitating defect prediction in software engineering. Created by A. Gunes Koru and used in several studies, this dataset includes various attributes related to software quality, encapsulation, complexity, and defectiveness.

Target Variable

`NUMDEFECTS`: This variable stores the number of defects recorded for a definition of class in an object-oriented programming code. It is the target variable used in predictive models to determine the likelihood of a class being defective.

Features at the Class Level

Class-level attributes store data that is shared among all the class instances. The features considered at a class level include:

- `PERCENT_PUB_DATA`: The percentage of public and protected data in a class. Lower values indicate better encapsulation.
- `ACCESS_TO_PUB_DATA`: The frequency of access to a class's public and protected data. Lower values indicate better encapsulation.
- `COUPLING_BETWEEN_OBJECTS`: The number of distinct, non-inheritance-related classes on which a class depends. High coupling may require more classes to be introduced together.
- `DEPTH`: The level of a class in its hierarchy. A greater depth indicates increased inheritance and complexity.
- `LACK_OF_COHESION_OF_METHODS`: For each data field in a class, it calculates the percentage of methods using that field. Lower percentages indicate higher cohesion.
- `NUM_OF_CHILDREN`: The number of classes derived from a specific class.
- `DEP_ON_CHILD`: A binary indicator of whether a class depends on a descendant.
- `FAN_IN`: The count of calls to a class by higher modules.

- `RESPONSE_FOR_CLASS`: The number of methods implemented in a class plus inherited methods. Lower values indicate higher polymorphism.
- `WEIGHTED_METHODS_PER_CLASS`: The count of methods implemented in a class, excluding inherited methods. Lower values indicate higher polymorphism.

Features at Method Level

Method-level attributes are metrics that are calculated at the level of individual methods (functions or procedures) within a software module. At the class level, these features are aggregated using four transformations: minimum (min), maximum (max), average (avg), and sum (sum), over all the methods in a class. The descriptions of these transformations are as follows:

- *min:* The minimum value across all methods in the class.
- *max:* The maximum value across all methods in the class.
- *avg:* The average value across all methods in the class.
- *sum:* The total value across all methods in the class.

The specific features considered at the method level include:

- `LOC_BLANK`: Lines containing only whitespace.
- `BRANCH_COUNT`: The number of branches in the module, indicating decision nodes.
- `LOC_CODE_AND_COMMENT`: Lines containing both code and comments.
- `LOC_COMMENTS`: Total number of lines in a module, including blank lines, comments, and source lines.
- `CYCLOMATIC_COMPLEXITY`: Measures the complexity of a module's decision structure via linearly independent paths.
- `DESIGN_COMPLEXITY`: Measures decision structure complexity related to module calls.
- `ESSENTIAL_COMPLEXITY`: Indicates the degree of unstructured constructs within a module.
- `LOC_EXECUTABLE`: Source lines containing only code and whitespace.
- `HALSTEAD_CONTENT`: Quantifies algorithm complexity independent of programming language.
- `HALSTEAD_DIFFICULTY`: Measures the difficulty level of understanding the program.
- `HALSTEAD_EFFORT`: Estimates the mental effort needed to develop the program.
- `HALSTEAD_ERROR_EST`: Estimates the number of potential errors in the program.
- `HALSTEAD_LENGTH`: Total number of operator and operand occurrences.
- `HALSTEAD_LEVEL`: Level at which the program can be understood.
- `HALSTEAD_PROG_TIME`: Estimated time to implement the algorithm.

- HALSTEAD_VOLUME: Minimum bits required to code the program.
- NUM_OPERANDS: Count of variables, identifiers, constants, and function names during calls.
- NUM_UNIQUE_OPERANDS: Count of unique variables, identifiers, constants, and function names.
- NUM_UNIQUE_OPERATORS: Count of unique operators used in the code.
- LOC_TOTAL: Total lines of code.

Example Calculation

To illustrate, consider the metric LOC_BLANK:

- minLOC_BLANK: The minimum number of blank lines among all methods in the class.
- maxLOC_BLANK: The maximum number of blank lines among all methods in the class.
- avgLOC_BLANK: The average number of blank lines among all methods in the class.
- sumLOC_BLANK: The total number of blank lines across all methods in the class.

Table 1 summarizes the attributes used in our dataset, indicating their value ranges. All attributes are numerical, providing a clear understanding of the dataset's structure and how various machine-learning techniques can be applied efficiently. Some attributes are represented with ranges of the form [x, x], which are redundant as they represent a single value. These attributes were kept in the dataset to preserve the original data structure and minimize alterations. To enhance clarity, such ranges have been simplified to a single value in the table where applicable.

Table 1: Attributes and their value ranges

Attribute	*Value Range*
ACCESS_TO_PUB_DATA	0
COUPLING_BETWEEN_OBJECTS	[0, 24]
DEPTH	[1, 7]
LACK_OF_COHESION_OF_METHODS	[0, 100]
NUM_OF_CHILDREN	[0, 5]
DEP_ON_CHILD	[0, 1]
FAN_IN	[0, 3]
RESPONSE_FOR_CLASS	[0, 222]
WEIGHTED_METHODS_PER_CLASS	[0, 100]
minLOC_BLANK	0
minBRANCH_COUNT	1

Attribute	*Value Range*
minLOC_CODE_AND_COMMENT	0
minLOC_COMMENTS	0
minCYCLOMATIC_COMPLEXITY	1
minDESIGN_COMPLEXITY	1
minESSENTIAL_COMPLEXITY	1
minLOC_EXECUTABLE	[0, 4]
minHALSTEAD_CONTENT	[0, 13.21]
minHALSTEAD_DIFFICULTY	[0, 2.5]
minHALSTEAD_EFFORT	[0, 76.08]
minHALSTEAD_ERROR_EST	[0, 0.01]
minHALSTEAD_LENGTH	[0, 10]
minHALSTEAD_LEVEL	[0, 0.67]
minHALSTEAD_PROG_TIME	[0, 4.23]
minHALSTEAD_VOLUME	[0, 31.7]
minNUM_OPERANDS	[0, 6]
minNUM_OPERATORS	[0, 6]
minNUM_UNIQUE_OPERANDS	[0, 5]
minNUM_UNIQUE_OPERATORS	[0, 5]
minLOC_TOTAL	[1, 10]
maxLOC_BLANK	[0, 58]
maxBRANCH_COUNT	[1, 89]
maxLOC_CODE_AND_COMMENT	[0, 12]
maxLOC_COMMENTS	[0, 44]
maxCYCLOMATIC_COMPLEXITY	[1, 45]
maxDESIGN_COMPLEXITY	[1, 45]
maxESSENTIAL_COMPLEXITY	[1, 26]
maxLOC_EXECUTABLE	[0, 262]
maxHALSTEAD_CONTENT	[0, 193.06]
maxHALSTEAD_DIFFICULTY	[0, 53.75]
maxHALSTEAD_EFFORT	[0, 324803.51]
maxHALSTEAD_ERROR_EST	[0, 2.64]
maxHALSTEAD_LENGTH	[0, 1106]
maxHALSTEAD_LEVEL	[0, 2]
maxHALSTEAD_PROG_TIME	[0, 18044.64]
maxHALSTEAD_VOLUME	[0, 7918.82]
maxNUM_OPERANDS	[0, 428]

Attribute	*Value Range*
maxNUM_OPERATORS	[0, 678]
maxNUM_UNIQUE_OPERANDS	[0, 120]
maxNUM_UNIQUE_OPERATORS	[0, 37]
maxLOC_TOTAL	[1, 288]
avgLOC_BLANK	[0, 13.105]
avgBRANCH_COUNT	[1, 21.167]
avgLOC_CODE_AND_COMMENT	[0, 1.917]
avgLOC_COMMENTS	[0, 11.545455]
avgCYCLOMATIC_COMPLEXITY	[1, 11.083]
avgDESIGN_COMPLEXITY	[1, 8.583]
avgESSENTIAL_COMPLEXITY	[1, 7.583]
avgLOC_EXECUTABLE	[0, 61]
avgHALSTEAD_CONTENT	[0, 56.528]
avgHALSTEAD_DIFFICULTY	[0, 19.182]
avgHALSTEAD_EFFORT	[0, 32843.282]
avgHALSTEAD_ERROR_EST	[0, 0.377]
avgHALSTEAD_LENGTH	[0, 187.833]
avgHALSTEAD_LEVEL	[0, 0.938]
avgHALSTEAD_PROG_TIME	[0, 1824.627]
avgHALSTEAD_VOLUME	[0, 1128.075]
avgNUM_OPERANDS	[0, 72.5]
avgNUM_OPERATORS	[0, 115.333]
avgNUM_UNIQUE_OPERANDS	[0, 32.772]
avgNUM_UNIQUE_OPERATORS	[0, 16.083]
avgLOC_TOTAL	[1, 79.818]
sumLOC_BLANK	[0, 510]
sumBRANCH_COUNT	[1, 1046]
sumLOC_CODE_AND_COMMENT	[0, 57]
sumLOC_COMMENTS	[0, 254]
sumCYCLOMATIC_COMPLEXITY	[1, 572]
sumDESIGN_COMPLEXITY	[1, 409]
sumESSENTIAL_COMPLEXITY	[1, 345]
sumLOC_EXECUTABLE	[0, 2313]
sumHALSTEAD_CONTENT	[0, 2865.06]
sumHALSTEAD_DIFFICULTY	[0, 1590.28]
sumHALSTEAD_EFFORT	[0, 1973866.93]

Attribute	*Value Range*
sumHALSTEAD_ERROR_EST	[0, 21.02]
sumHALSTEAD_LENGTH	[0, 11172]
sumHALSTEAD_LEVEL	[0, 32.51]
sumHALSTEAD_PROG_TIME	[0, 109659.3]
sumHALSTEAD_VOLUME	[0, 63240.96]
sumNUM_OPERANDS	[0, 4439]
sumNUM_OPERATORS	[0, 6733]
sumNUM_UNIQUE_OPERANDS	[0, 1820]
sumNUM_UNIQUE_OPERATORS	[0, 1383]
sumLOC_TOTAL	[1, 2883]
NUMDEFECTS	[0, 101]

2.2 EXPERIMENTAL SETUP

2.2.1 Data Preprocessing

Data preprocessing is an essential step in preparing data for machine learning algorithms. The following key steps were applied to preprocess the dataset derived from the ARFF file `kc1-class-level-numericdefect.arff`:

1. **Loading the Data**
   - The dataset was loaded from an ARFF file using the `arff` module from the SciPy library.
   - The data was then converted into a Pandas DataFrame for easier manipulation and analysis.
2. **Removing Duplicates**
   - Duplicate rows in the dataset were identified using the `duplicated` method in Pandas, and a total of four duplicates were removed using the `drop_duplicates` method.
3. **Handling Missing Values**
   - Missing values were addressed using the `SimpleImputer` from the Scikit-Learn library.
   - The imputation method was set to `mean`, which fills in missing values with the average value of the respective feature.
4. **Excluding Outliers**

- Outliers were identified and excluded using the `IsolationForest` algorithm.
- The contamination parameter was set to 0.1, indicating the expectation of 10% of the data being outliers.
- A total of 14 outliers were identified and removed from the dataset.

5. **Scaling the Data**
   - Feature scaling was performed using the `StandardScaler` from Scikit-Learn.
   - The data was standardized to a mean of 0 and a standard deviation of 1, which enhanced model performance and sped up convergence.

2.2.2 Training and Testing Procedure

In this study, two types of cross-validation techniques were utilized to evaluate the performance of the machine learning models: leave-one-out and k-fold cross-validation. These methods were used to ensure the robustness and reliability of the models by assessing their performance on different subsets of the data. Specifically, the dataset was trained and tested using the following four methods:

1. Leave-one-out, outliers included
2. Leave-one-out, outliers excluded
3. K-fold Cross-Validation, outliers included
4. K-fold Cross-Validation, outliers excluded

The value of k in k-fold cross-validation varied depending on the algorithm used. More details about the specific values of k are provided in the Results chapter.

K-fold Cross-Validation

In k-fold cross-validation, the dataset is partitioned into k equally sized folds. Each iteration uses one segment as the test set, with the other $k - 1$ segments serving as the training set. This process is repeated k times, with each fold being used exactly once as the test set. The performance metrics from all iterations are then averaged to produce a single performance measure.

Using these cross-validation techniques, the models were trained and tested on both the original dataset and the dataset with outliers excluded. This comprehensive approach ensures that the models' performance is thoroughly evaluated, accounting for the potential impact of outliers and providing robust estimates of their predictive accuracy.

Leave-One-Out

Leave-one-out is a special case of k-fold cross-validation where k is equal to the number of instances in the dataset. In each iteration, one instance is used as the test set while the remaining observations form the training set. This process is repeated until every instance has been used as a test set exactly once. The following Python code snippet illustrates the implementation of the leave-one-out method:

```
def loocv(X, y, model):
    # Initialize Leave-One-Out cross-validation
    loo = LeaveOneOut()

    # Loop over each train-test split in Leave-One-Out cross-validation
    for train_index, test_index in loo.split(X):

        # Split the dataset into training and testing sets
        # based on the current indices
        X_train, X_test = X.iloc[train_index], X.iloc[test_index]
        y_train, y_test = y.iloc[train_index], y.iloc[test_index]

        # Fit the model on the training data
        model.fit(X_train, y_train)

        # Predict the target value for the test data
        y_pred = model.predict(X_test)
```

2.2.3 Hyperparameter Tuning

Hyperparameter tuning is a crucial step in optimizing machine learning models to achieve better performance on a given task. In this section, we discuss the methods utilized to find optimal hyperparameters for our models, specifically focusing on the use of GridSearchCV from the scikit-learn library.

Hyperparameters are parameters of a model whose values are set before the learning process begins. They can significantly impact the model's ability to learn from the data. Tuning these hyperparameters involves finding the optimal set that minimizes a chosen metric, such as mean squared error (MSE), mean absolute error (MAE), or maximizes the coefficient of determination (R^2).

GridSearchCV performs an exhaustive search over a specified parameter grid, evaluating each setup using cross-validation (CV). This approach ensures that our model's performance

metrics are robust and not overly optimistic or pessimistic due to the choice of training and validation splits.

2.2.4 Feature Importance

To understand which features have the most significant impact on the model's predictions, we analyzed the importance of the feature using a Gradient Boosting Regressor. We chose Gradient Boosting for its ability to produce highly accurate models, especially with complex datasets, by sequentially building an ensemble of trees where each new tree corrects the errors of the previous ones. This method leverages the Mean Decrease in Impurity (MDI) to calculate feature importance [11]. MDI is well-suited for machine learning algorithms that use decision trees or ensembles of trees [13]. Since Gradient Boosting is an ensemble of trees, it effectively provides a robust measure of feature importance, allowing us to identify which features contribute most significantly to the model's predictions.

Using the trained model, we extracted the 15 most important features, shown in Table 2. More information about the hyperparameters on which the algorithm was tuned can be found in the section on the gradient boosting algorithm. The algorithm determines the most important features by assessing how much each feature contributes to reducing impurity across all trees in the ensemble. Given the large number of features in the dataset, we chose to display only the top 15 most important ones to provide a representative subset of the most influential features. This approach emphasizes the features with the greatest impact on the model's predictions, enhancing both interpretability and decision-making.

Table 2: Top 15 most important features

Feature	Importance
sumLOC_TOTAL	0.0422
sumBRANCH_COUNT	0.0346
COUPLING_BETWEEN_OBJECTS	0.0340
maxHALSTEAD_VOLUME	0.0326
maxLOC_TOTAL	0.0296
sumHALSTEAD_PROG_TIME	0.0285
maxNUM_UNIQUE_OPERANDS	0.0255
sumCYCLOMATIC_COMPLEXITY	0.0249
avgHALSTEAD_EFFORT	0.0244
LACK_OF_COHESION_OF_METHODS	0.0230
sumHALSTEAD_DIFFICULTY	0.0228
sumHALSTEAD_VOLUME	0.0215
maxHALSTEAD_LENGTH	0.0208
maxHALSTEAD_EFFORT	0.0206
maxLOC_COMMENTS	0.0205

The significance of a feature in a Gradient Boosting Regressor, referred to as Mean De-

crease Impurity (MDI), is determined by the (normalized) total reduction in the criterion (such as the Gini index or variance) that the feature contributes [13]. MDI provides a cumulative importance score across all trees in the ensemble, offering a more comprehensive assessment than individual trees. In regression tasks, the impurity measure typically used is *variance*.

A tree is constructed from a learning sample of size N drawn from $P(X_1, \ldots, X_p, Y)$ using a recursive procedure which identifies at each node t the split $s_t = s^*$ for which the partition of the N_t node samples into t_L and t_R maximizes the decrease

$$\Delta i(s,t) = i(t) - p_L i(t_L) - p_R i(t_R) \tag{2.1}$$

of some impurity measure $i(t)$ (e.g., the Gini index, the Shannon entropy, or the variance of Y), and where $p_L = \frac{N_{tL}}{N_t}$ and $p_R = \frac{N_{tR}}{N_t}$ are the proportions of samples in the left and right child nodes after the split. The construction of the tree stops, e.g., when nodes become pure in terms of Y or when all variables X_i are locally constant [13].

The notation $s_t = s^*$ means that the split s_t at node t is chosen to be the optimal split s^*. This optimal split is the one that maximizes the decrease in impurity at that node. The goal is to find the split s^* that maximizes the decrease in impurity, making the data in each resulting child node t_L (left child) and t_R (right child) more homogeneous (less impure) compared to the parent node t [13].

The formula for variable importance in an ensemble of trees is given by:

$$\text{Imp}(X_m) = \frac{1}{N_T} \sum_T \sum_{t \in T : v(s_t) = X_m} p(t) \Delta i(s_t, t) \tag{2.2}$$

where $p(t)$ is the proportion N_t/N of samples reaching t and $v(s_t)$ is the variable used in split s_t [13].

2.3 THE BASELINE MODEL

A baseline model provides a simple reference point against which the performance of more complex models can be compared. It is a benchmark to determine whether the more sophisticated models improve predictive performance. If the advanced models cannot outperform the baseline, it suggests that they may be overfitting or not adequately capturing the underlying patterns in the data.

In this study, we implemented a baseline model using the scikit-learn framework. The baseline model, implemented as a custom estimator, predicts the mean value of the target variable for all instances, that is $\hat{y}_i = \overline{y} \quad$ for all i, where $\hat{y}$ is the predicted value and $\overline{y}$ the average over all i.

If the results of our advanced models are significantly better than the baseline, it indicates that these models are effectively learning from the features and capturing the underlying patterns in the data. Conversely, if the performance is comparable to or worse than the baseline, the models may not be leveraging the feature information effectively, or they might be overfitting.

2.4 MACHINE LEARNING ALGORITHMS USED

Machine learning involves an automated approach to identifying patterns within data [8]. In supervised machine learning, we create models to perform predictive analysis by learning the relationship between a set of *descriptive features* and a *target feature* from historical data or instances. Once trained, these models can be used to predict outcomes for new instances [8].

Supervised machine learning problems are generally divided into two categories: *regression* and *classification* [8]. Regression problems are concerned with predicting a continuous value. The objective is to identify the underlying function that generates the values, which involves fitting a mathematical function to a curve that approximates the data points as closely as possible [14]. In contrast, classification problems aim to predict a discrete label. The task is to categorize each instance into one of several predefined categories based on its descriptive features.

Since we aim to predict continuous values, specifically, the number of defects (NUMDEFECTS) in software modules, we utilize *regression algorithms* in this study. Regression algorithms automate the process of learning a model that maps the input features to the continuous target variable. In this study, we utilized various machine learning algorithms to predict the need for maintenance. The machine learning algorithms that we used are:

- Artificial Neural Networks,
- Support Vector Regressor,
- Linear Regression,
- Decision Tree Regressor.

2.4.1 Artificial Neural Networks

Neural networks are a computational model inspired by the way the human brain is structured. They are made up of interconnected nodes (neurons) that process information through multiple layers. The grand vision of neural networks is to create artificial intelligence by building machines whose architecture simulates the computations in the human nervous system [1]. In an artificial neural network, each neuron's input is scaled by a weight, which

affects the neuron's output. The network computes a function by passing values from input neurons through weights to output neurons. Similar to how biological organisms learn from external stimuli, ANNs learn from training data that consists of input-output pairs for the target function. By adjusting the weights between neurons based on this data, the network improves its ability to predict accurately. This iterative adjustment process enhances the network's performance on new, unseen data, a concept known as model generalization. The key benefit of machine learning models is their ability to apply knowledge from training data to new examples [1].

Although there are multiple types of neural networks, such as artificial, convolution, and recurrent neural networks [15], we have decided to focus on Multilayer Perceptron (MLP) which is the most basic type of ANN and will proceed by discussing the architecture and functioning of ANNs in detail.

The Architecture of Artificial Neural Networks

Neural networks can have a single-layer or a multi-layer architecture. In this thesis, we will focus on a multi-layer neural network. Multi-layer neural networks extend the basic architecture of single-layer networks by incorporating one or more hidden layers between the input and output layers. These hidden layers are composed of neurons that perform intermediate computations, making the network more powerful and capable of learning more complex functions. Unlike single-layer perceptrons, where the input layer directly maps to the output, multi-layer networks utilize a feed-forward architecture where each layer passes its output to the next layer in the sequence [1].

Consider an example of a single-layer ANN that predicts binary class variable $y \in \{-1, +1\}$. The input layer contains d nodes that transmit the d features $\overline{\mathbf{X}} = [x_1, \ldots, x_d]$ with edges of weight $\overline{\mathbf{W}} = [w_1, \ldots, w_d]$ to an output layer that consists of only one node. The input layer does not perform any computation. The linear function $\overline{\mathbf{W}} \cdot \overline{\mathbf{X}} = \sum_{i=1}^{d} w_i x_i$ is computed at the output node. Subsequently, the sign of this real value is used to predict the dependent variable of $\overline{\mathbf{X}}$. Therefore the prediction $\hat{y}$ is given by:

$$\hat{y} = \text{sign}\left\{\overline{\mathbf{X}} \cdot \overline{\mathbf{W}}\right\} = \text{sign}\left\{\sum_{j=1}^{d} w_j x_j\right\} \tag{2.3}$$

where w_j are the weights and x_j are the input features.

Multilayer neural networks consist of multiple computational layers. The layers beyond the input and output layers are known as hidden layers because their computations are not directly observable to the user. This type of network is termed a feed-forward network, as the layers are arranged such that each layer feeds into the next in a forward direction from input to output. An example is illustrated in Figure 1.

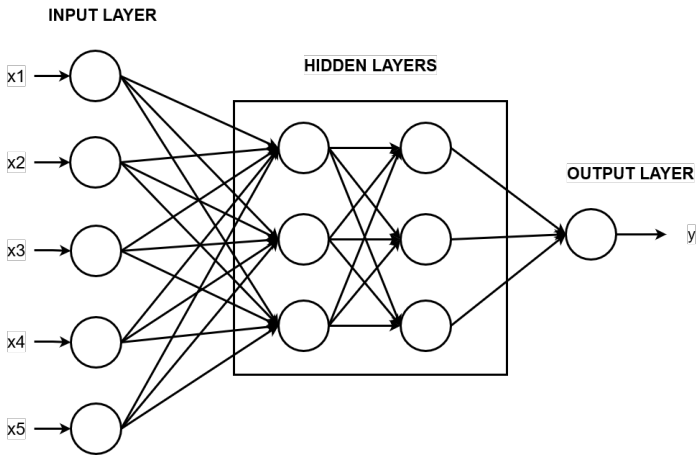

Figure 1: The fundamental architecture of a feed-forward network [1].

Activation Function

The activation function determines the output based on a specific input or set of inputs. The choice of activation functions significantly affects the learning process. Common activation functions are *identity*, *sign*, *sigmoid*, *tahn*, *hyperbolic tangent* (hard tanh), and *rectified linear unit* (ReLU), each with its characteristics and applications. For multi-layered architectures nonlinear activation functions are used, such as ReLU, sign, tahn, etc. We use the notation Φ to denote the activation function:

$$\hat{y} = \Phi(W \cdot X) \tag{2.4}$$

The break-up of neuron computations into two distinct values is illustrated in Figure 2 [1].

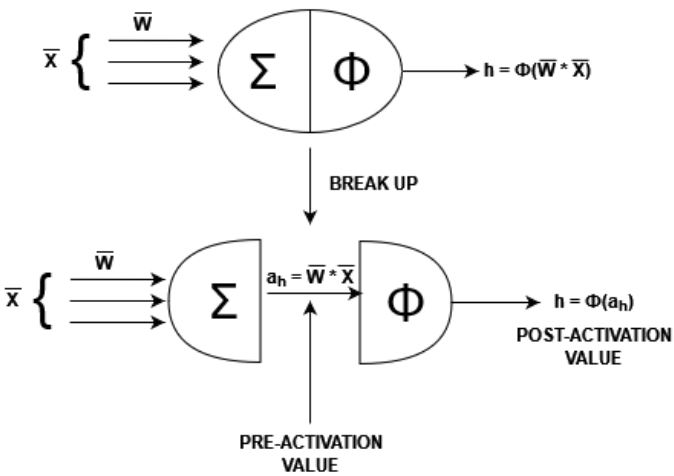

Figure 2: Pre-activation and post-activation values within a neuron [1].

In this study, we use the Rectified Linear Unit (ReLU) activation function, which is defined as:

$$\Phi(v) = \max\{v, 0\} \tag{2.5}$$

ReLU is preferred for its nonlinearity, allowing neural networks to effectively capture complex patterns. It addresses the vanishing gradient problem by providing a constant gradient for positive inputs. Because of its simplicity, it is fast to compute and promotes sparsity in activations, enhancing the generalization and scalability of the neural network model. A quick summary of how the training algorithm works can be given as follows, as shown in [4].

1. An input vector is fed to the input nodes.
2. The inputs are fed forward through the network (Figure 1):
   - The inputs and the first-layer weights (here labeled as w) are used to decide whether the hidden nodes fire or not.
   - The outputs of these neurons and the second-layer weights are used to decide if the output neurons fire or not.
3. The *error* is computed according to the loss function, typically MSE or MAE for regression models.
4. This error is propagated backward through the network to update the weights with a certain learning rate, to accommodate the prediction to the last learning batch.

Hyperparameter Tuning

Neural networks in this study are implemented using the MLPRegressor class from the scikit-learn Python package. The script initializes multiple MLPRegressor instances, each configured with different hidden layer architectures specified in the $hidden_layer_configs$ variable. These configurations include setups such as (300), (500), (200, 100), and (300, 200, 100), where each number represents the number of neurons in a hidden layer and *n* denotes the number of hidden layers. The *activation* function is set to 'relu', which introduces nonlinearity and aids in learning complex patterns while mitigating vanishing gradient problems. The *solver* chosen is 'adam', an adaptive learning rate optimization algorithm that is effective for large datasets and high-dimensional problems. Regularization is controlled with an *alpha* value of 0.0003, which helps prevent overfitting by penalizing large weights. Training is allowed to run for a maximum of 1500 iterations to ensure thorough convergence. The *random_state* is set to 42 to ensure reproducibility of the results across different runs [20].

This systematic approach allows for exploring various neural network architectures and hyperparameter configurations, facilitating the identification of the optimal setup for accurate regression predictions.

2.4.2 Support Vector Regressor

Support Vector Regression (SVR) is an extension of the Support Vector Machine (SVM) for regression tasks. While SVMs are primarily designed for binary classification, SVR adapts these principles to predict continuous-valued outputs.

SVMs aim to find the optimal hyperplane that maximizes the margin between two classes in a binary classification setting. This optimization problem is characterized by the use of kernels, sparse solutions, and Vapnik-Chervonenkis (VC) control of the margin and the number of support vectors [3]. Similarly, SVR utilizes these features but reformulates the problem to handle regression.

SVM generalization to SVR is accomplished by introducing an ϵ-insensitive region around the function, called the ϵ-tube. This tube reformulates the optimization problem to identify the tube that best approximates the continuous-valued function, balancing model complexity with prediction error. Specifically, Support Vector Regression (SVR) is framed as an optimization problem by first defining a convex ϵ-insensitive loss function to minimize, and then identifying the flattest tube that encompasses most of the training instances. Consequently, a multi-objective function is constructed from both the loss function and the geometric properties of the tube. Convex optimization, which guarantees a unique solution, is then applied using suitable numerical optimization algorithms. The hyperplane is defined by support vectors, which are training samples that lie outside the tube's boundary. Similar to Support Vector Machines (SVM), in Support Vector Regression (SVR), the support vectors are the most influential samples that shape the tube. Moreover, in a supervised learning context, it is assumed that the training and test data are independent and identically distributed (iid), drawn from the same fixed but unknown probability distribution [3].

The primary advantage of SVR is that its computational complexity is independent of the dimensionality of the input space. This characteristic makes SVR especially well-suited for high-dimensional data. Additionally, SVR exhibits excellent generalization capabilities, providing high prediction accuracy even with complex datasets [3].

Figure 3 illustrates the concept of SVR, where the middle solid line represents the function $f(x) = \langle w, x \rangle + b$, and the other two lines represent the ϵ-tube around it. The points outside this tube contribute to the error and are penalized based on the slack variables ξ_i and ξ_i^* [3]. The real-valued function being approximated can be expressed as shown in Equation 2.6. For multidimensional data, we extend $\mathbf{x}$ by adding a constant term and integrate $\mathbf{b}$ into the $\mathbf{w}$ vector to streamline the mathematical representation, resulting in the multivariate regression shown in Equation 2.7.

$$y = f(x) = \langle w, x \rangle + b \tag{2.6}$$

$$f(x) = \begin{bmatrix} w \\ b \end{bmatrix}^T \begin{bmatrix} x \\ 1 \end{bmatrix} = w^T x + b \quad x, w \in \mathbb{R}^{M+1} \tag{2.7}$$

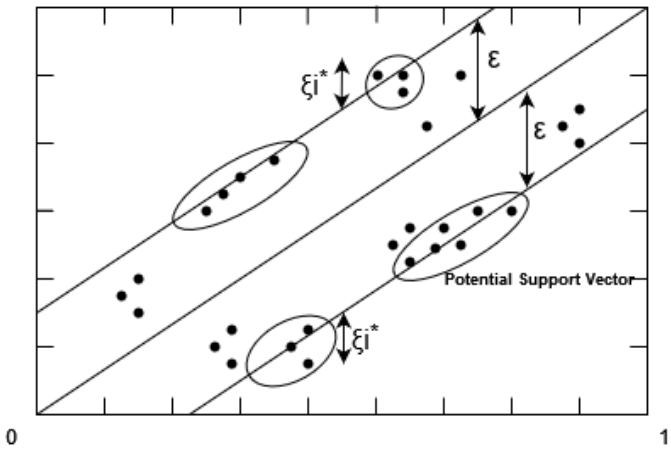

Figure 3: One-dimensional linear SVR [3].

SVR addresses this function approximation issue as an optimization challenge, aiming to identify the narrowest tube that fits around the surface while reducing the prediction error, which represents the difference between the predicted and actual outputs.

Loss Function

The goal is to minimize the discrepancy between the predicted value of the function for a given input and the actual output. SVR adopts an ϵ-insensitive loss function, penalizing predictions that are farther than ϵ from the desired output. The value of ϵ determines the width of the tube: a smaller ϵ signifies a lower tolerance for error, which also impacts the number of support vectors and, thus, the sparsity of the solution. This effect can be illustrated by Figure 3. When ϵ is reduced, the boundary of the tube shifts inward. As a result, more data points are situated near the boundary, which signifies a higher number of support vectors. Similarly, increasing ϵ will result in fewer points around the boundary [3].

Because it is less sensitive to noisy inputs, the ϵ-insensitive region makes the model more robust. Several loss functions can be adopted, including the linear, quadratic, and Huber ϵ, as are respectively defined as follows, as specified in [3]:

$$L_\epsilon(y, f(x, w)) = \begin{cases} 0 & \text{if } |y - f(x), w| \leq \epsilon \\ |y - f(x, w)| - \epsilon & \text{otherwise} \end{cases} \tag{2.8}$$

$$L_\epsilon(y, f(x, w)) = \begin{cases} 0 & \text{if } |y - f(x, w)| \leq \epsilon \\ (|y - f(x, w)| - \epsilon)^2 & \text{otherwise} \end{cases} \tag{2.9}$$

$$L_\epsilon(y, f(x,w)) = \begin{cases} c|y - f(x,w)| - \frac{c^2}{2} & \text{if } |y - f(x)| > c \\ \frac{1}{2}|y - f(x,w)|^2 & |y - f(x,w)| \leq c \end{cases} \tag{2.10}$$

The choice of a loss function is influenced by prior knowledge of the noise distribution affecting the data, the desired level of model sparsity, and the computational complexity of training. The loss functions in question are symmetric and convex. While asymmetrical loss functions can be used to address specific issues of underestimation or overestimation, convex loss functions are preferred because they guarantee that the optimization problem has a unique solution that can be determined within a finite number of steps [3].

Hyperparameter Tuning

Support Vector Regression (SVR) is implemented using the SVR class from the scikit-learn Python package. The *kernel* parameter is set to 'rbf' (Radial Basis Function), which allows the model to handle non-linear relationships by mapping data into a higher-dimensional space where a hyperplane can be effectively used for regression. The *C* parameter, set to 10, is a regularization parameter that controls the trade-off between achieving a low training error and minimizing the model complexity. A higher value of *C* places more emphasis on minimizing the training error by allowing the model to fit the training data more closely. This can lead to a more complex model that might overfit the training data. Conversely, a lower value of *C* increases the tolerance for errors, which may lead to a simpler model with potentially higher bias but better generalization to new data. The *epsilon* parameter, set to 0.1, defines the width of the epsilon-insensitive tube within which no penalty is applied to errors. This helps manage the trade-off between model accuracy and robustness, allowing predictions within 0.1 units of the actual values to be considered acceptable without contributing to the loss function. This configuration enables the SVR model to balance fitting the data accurately while maintaining robustness to noise and non-linear patterns [21].

2.4.3 Linear Regression

Linear regression is utilized to determine the best-fitting linear equation that describes the relationship between the dependent variable y and the independent variables $x_1, x_2, \ldots, x_p$. This is achieved by estimating the coefficients $\beta_0, \beta_1, \ldots, \beta_p$ such that the sum of squared residuals between the observed values and the values predicted by the model is minimized.

When there is only one independent variable, the model is referred to as *simple linear regression*; when there are multiple independent variables, it is called *multiple linear regression* [7].

Model Formulation

In its simplest form, the linear regression model assumes that the relationship between y and $\mathbf{x}$ is linear:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p + \epsilon \tag{2.11}$$

where:

- y is the dependent variable (response or target),
- $x_1, x_2, \ldots, x_p$ represent the independent variables (predictors or features),
- β_0 is the intercept,
- $\beta_1, \beta_2, \ldots, \beta_p$ are the regression coefficients that represent the effect of each predictor on y. The regression coefficient β_1 indicates how much the predicted value of y changes when x_1 increases by one unit. If β_1 is positive, an increase in x_1 will lead to an increase in y. If β_1 is negative, an increase in x_1 will result in a decrease in y,
- ϵ is the error, which captures the variability in y that cannot be explained by the predictors.

Hyperparameter Tuning

Linear Regression is implemented using the LinearRegression class from the scikit-learn Python package. The *copy_X* parameter is set to True, which indicates that a copy of the input data will be made before fitting the model. This ensures that the original data remains unmodified during the training process. The *fit_intercept* parameter is set to False, meaning that the model will not include an intercept term in the regression equation. This is typically chosen when the data is centered around zero or when the intercept is not needed for the specific analysis. The *n_jobs* parameter is set to 1, specifying the number of CPU cores to use for computation. A value of 1 indicates that the computation will be performed using a single core. Finally, the *positive* parameter is set to True, which enforces that the coefficients of the regression model are non-negative. This constraint can be useful when it is known a priori that the coefficients should be positive due to the nature of the problem [19].

2.4.4 Decision Tree Regressor

A Decision Tree Regressor is a powerful and intuitive tool for predictive modeling, particularly suited for tasks involving the prediction of continuous outcomes. In the context of this thesis, where the objective is to predict the number of defects in software modules, decision tree regression offers several advantages. We decided to use this model due to its simplicity in interpretation, ability to handle numerical data, and robustness to outliers.

A decision tree consists of:

- *Internal nodes*: Represent decision points based on features.
- *Branches*: The segments of the trees that connect the nodes.
- *Terminal nodes or leaves*: Terminal nodes that predict the outcome.

Algorithm

The decision tree regressor algorithm can be systematically described through the following two key steps:

1. The predictor space, which consists of all possible values for $X_1, X_2, \ldots, X_p$, is divided into J distinct and non-overlapping regions, $R_1, R_2, \ldots, R_J$.
2. For each observation that falls within region R_j, the prediction is made by taking the mean of the response values of the training observations in R_j.

Constructing the Regions

The goal is to identify regions $R_1, \ldots, R_J$ that minimize the Residual Sum of Squares (RSS), which is defined as:

$$\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

where $\hat{y}_{R_j}$ denotes the average response for the training observations within the j-th region. Given the computational difficulty of evaluating every possible partition of the feature space into J regions, we employ a top-down, greedy approach known as *recursive binary splitting* [7].

Recursive Binary Splitting

Recursive binary splitting is a top-down approach that begins at the root of the tree (where all observations are in a single region) and successively splits the predictor space into two branches. The method is greedy because, at each step, it chooses the split that results in the greatest immediate reduction in RSS, without considering future splits.

To perform recursive binary splitting, we first select the predictor X_j and the cutpoint s that results in the greatest reduction in RSS. Specifically, we examine all predictors $X_1, \ldots, X_p$ and all possible cutpoints s for each predictor, and choose the combination that minimizes RSS:

$$\text{RSS}_{\text{split}} = \sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

where $R_1(j, s) = \{X \mid X_j < s\}$ and $R_2(j, s) = \{X \mid X_j \geq s\}$. Here, $\hat{y}_{R_1}$ and $\hat{y}_{R_2}$ are the mean responses for the training observations in $R_1(j, s)$ and $R_2(j, s)$, respectively. This process is repeated, further splitting the regions to minimize RSS until a predefined stopping criterion is met (e.g., a minimum number of observations per region) [7].

Tree Pruning to Avoid Overfitting

The described method may produce excellent predictions on the training set but can overfit the data, resulting in poor performance on new data. Overfitting happens when a model becomes too complex and captures noise rather than the underlying pattern. To prevent overfitting, we employ tree pruning [7].

Tree pruning involves first growing a large tree and then trimming it to balance complexity and fit [7]. We use cost complexity pruning (also known as weakest link pruning), which generates a sequence of trees indexed by a nonnegative tuning parameter α. The objective is to select the subtree that minimizes the test error.

The pruning process is as follows:

1. Grow an extensive tree T_0 using the recursive binary splitting method.
2. For a given α, define the cost complexity criterion:

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

   where $|T|$ denotes the number of terminal nodes in the tree T, and R_m represents the regions defined by the terminal nodes.
3. Choose the subtree T_α that minimizes $C_\alpha(T)$.
4. Employ cross-validation to determine the optimal value of α that minimizes the test error.
5. Prune the large tree T_0 to derive the subtree T_α using the optimal α.

Pruning the tree reduces its complexity, decreases variance, and enhances its ability to generalize new data. This method ensures that the final model is both interpretable and effective in predicting the number of defects in software modules.

Hyperparameter Tuning

The decision tree is implemented using the DecisionTreeRegressor class from the scikit-learn Python package. The *ccp_alpha* parameter is set to 0.01, which is used for pruning the tree by specifying the complexity parameter. This helps to avoid overfitting by removing nodes that provide minimal improvement in model complexity. The chosen subtree will be

the one with the largest cost complexity that is smaller than ccp_alpha. The *criterion* is set to 'absolute_error', which measures the quality of a split using the mean absolute error, a robust measure against outliers. The *max_depth* is set to 10, limiting the maximum depth of the tree to prevent overfitting and ensure the model remains interpretable. The *max_features* parameter is set to 'auto', meaning that a subset of features is considered when looking for the best split. The *max_leaf_nodes* is set to 10, which limits the number of leaf nodes, further controlling the complexity of the model. The *min_impurity_decrease* is set to 0.01, ensuring that a split is only created if it decreases the impurity by at least this amount. The *min_samples_leaf* is set to 1, indicating that a leaf node must contain at least one sample. The *min_samples_split* is set to 5, which is the minimum number of samples required to split an internal node. Finally, the *splitter* is set to 'random', which means that the algorithm will randomly choose the best split among the top features, potentially enhancing model robustness by preventing overfitting to specific data patterns [18].

2.5 ENSEMBLE METHODS

Ensemble methods involve training multiple learners to address the same problem, in contrast to traditional approaches that construct a single learner from training data [22]. By combining multiple learners, ensemble methods typically achieve better generalization than single learners. These methods, also known as *committee-based learning* or *multiple classifier systems*, can utilize various algorithms like decision trees or neural networks to generate base learners.

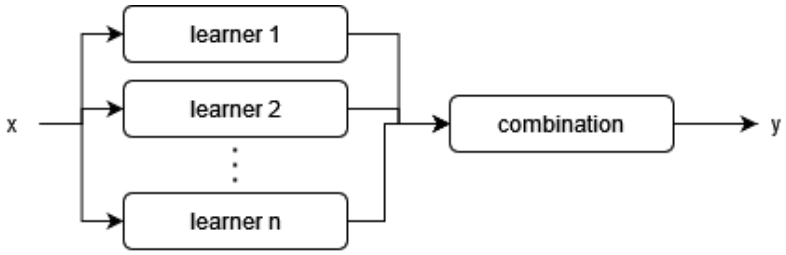

Figure 4: Common ensemble architecture.

Figure 4 shows a common ensemble architecture. An ensemble typically consists of multiple learners, called base learners, generated from training data using a base learning algorithm. Most ensemble methods use a single algorithm to create homogeneous base learners, leading to homogeneous ensembles. However, some methods employ different algorithms to produce heterogeneous learners, resulting in heterogeneous ensembles.

The primary appeal of ensemble methods is their ability to enhance the performance of weak learners, transforming them into strong learners capable of making highly accurate predictions. The boosting of weak learners is a fundamental advantage of ensemble methods.

Constructing an ensemble generally involves two steps: generating the base learners and combining them. A successful ensemble requires base learners to be both accurate and diverse. Interestingly, the computational cost of creating an ensemble is comparable to generating multiple versions of a single learner for model selection or parameter tuning [22].

Given the potential of ensemble methods to enhance predictive accuracy, we decided to implement them in our research to evaluate their performance against traditional algorithms. Success in this approach could result in improved accuracy in predicting the number of defects in software modules.

Bagging, random forests, and boosting leverage decision trees to create more powerful predictive models [7]. For this study, we opted for Random Forest and Gradient Boosting Regressor. Random Forest was selected due to its capability to mitigate overfitting by averaging the results of multiple decision trees, thereby enhancing accuracy and robustness. Gradient Boosting Regressor was chosen for its proficiency in improving performance by sequentially combining weak learners into a strong predictive model.

2.5.1 Random Forest

Random Forest (RF) is a powerful ensemble learning method that combines multiple decision trees to improve predictive performance and robustness. It aggregates predictions from a collection of decision trees, each trained on a random subset of the training data and features.

Model Structure

A Random Forest is composed of an ensemble of decision trees, where:

- Each tree is trained independently on a bootstrap sample (randomly sampled with replacement) from the training data.
- At each node of the tree, a random subset of features is selected for splitting. Usually, $\sqrt{p}$ features are selected for classification, and $\frac{p}{3}$ features are chosen for regression, where p is the total number of features.
- Predictions are made by aggregating the predictions of all trees (e.g., averaging for regression, voting for classification).

Similar to bagging, random forests build multiple decision trees on bootstrapped training samples. However, during the construction of these trees, each time a split is considered, a random sample of m predictors is chosen from the full set of p predictors as candidates for the split. The split is restricted to using only one of these m predictors. This process is repeated at each split, and typically $m \approx \sqrt{p}$, meaning the number of predictors considered at each split is approximately the square root of the total number of predictors [7].

Hyperparameter Tuning

A grid search was conducted to identify the optimal hyperparameters to enhance the performance of the Random Forest model. The grid search evaluated various combinations of parameters to minimize the loss function.

The *max_depth* parameter is set to *None*, allowing the trees to grow until they reach pure sets in the leaves or meet other stopping criteria. This setting maximizes tree depth flexibility but should otherwise be balanced with other parameters to avoid overfitting. The *max_features* parameter is set to 'log2', which means that at each split, only $\log_2(n_features)$ features are considered. This approach helps in decreasing the correlation between trees, thereby lowering model variance. The *min_impurity_decrease* parameter is set to 0.1, meaning that a split is only made if it results in a reduction of impurity by at least 0.1. This criterion ensures that splits contribute significantly to improving model performance. The *min_samples_leaf* parameter is set to 4, ensuring that leaf nodes have a minimum of 4 samples, which helps in preventing overfitting by avoiding overly specific splits. The *min_samples_split* parameter is set to 10, requiring at least 10 samples to split an internal node, thus promoting more robust tree structures. Finally, *n_estimators* is set to 100, providing a sufficient number of trees to capture data patterns effectively while maintaining manageable training times [12].

2.5.2 Gradient Boosting Regressor

Boosting is an effective ensemble method aimed at enhancing the predictive performance of decision trees. Unlike bagging, which builds multiple independent decision trees and combines their predictions, boosting constructs trees sequentially, each new tree attempts to correct the errors of the previous ones [7]. This iterative process allows boosting to create a strong predictive model by focusing on the mistakes made by earlier models.

In the context of predicting software module defects, we employ the *Gradient Boosting Regressor* (GBR), a specific implementation of boosting for regression tasks.

The primary parameters influencing the performance of the Gradient Boosting Regressor are:

- *Number of Trees (B):* This parameter determines the total number of trees in the ensemble. Boosting can overfit if B is too large. Cross-validation is typically used to select an optimal B [7].
- *Shrinkage Parameter (λ):* Also known as the learning rate, this small positive number controls the contribution of each tree to the final model. Common values are 0.01 or 0.001. A smaller λ requires a larger number of trees B to achieve good performance [7].

- *Number of Splits (d):* This controls the complexity of each tree. Setting $d = 1$ results in decision stumps, which lead to an additive model. Generally, d controls the interaction depth, indicating the maximum number of variables involved in each split.

The Algorithm

Gradient boosting is a powerful machine-learning technique that sequentially builds additive regression models. The process begins with an initial model, which can be a simple constant or a basic prediction function. This model provides a starting point for further improvement. The essence of gradient boosting lies in fitting additional models to the residuals, also known as pseudo-residuals, of the current model [7].

Pseudo-residuals are calculated as the gradient of the loss function with respect to the model's predictions. They represent how much each prediction deviates from the true value and thus indicate the direction and magnitude of the errors in the current model's predictions. These residuals are evaluated at each iteration, reflecting the areas where the model's performance is lacking.

In each iteration, a new model, referred to as the base learner or weak learner, is fitted to these pseudo-residuals. This base learner is typically a simple model, such as a decision tree with limited depth. The base learner's role is to capture and correct the errors made by the current model, focusing specifically on reducing the residuals.

The predictions of the base learner are then combined with those of the existing model to produce a new, updated model. This combination is done in an additive manner, meaning that each new model contributes incrementally to the overall prediction. The strength of each base learner is controlled by a parameter known as the learning rate or shrinkage, which determines how much weight is given to the new model's predictions relative to the existing model.

This iterative process continues until a predefined number of iterations is reached or until no significant improvement is observed. By focusing on the residuals and gradually refining the predictions, gradient boosting effectively improves model accuracy and robustness [6].

Hyperparameter Tuning

A grid search was conducted to identify the best hyperparameters to optimize the performance of the Gradient Boosting Regressor. The grid search evaluated various combinations of parameters to minimize the loss function.

The learning rate of 0.01 ensures that the model learns at a steady pace, while the absolute error loss function is robust to outliers. A maximum depth of 3 for the trees helps maintain simplicity and interpretability. Using 'log2' for maximum features allows the model to consider a subset of features at each split, which helps reduce variance. A minimum of 15

samples is required to split an internal node, which helps in reducing overfitting. The number of estimators is set to 300, ensuring that the model has enough trees to learn effectively. A tolerance of 0.01 provides that the training stops when further improvement is minimal. Finally, the *min_impurity_decrease* parameter set to 0.1 provides that a split is only made if it results in a significant reduction in impurity, promoting better generalization [11].

2.6 Evaluation Metrics

Evaluation metrics are used to assess the performance of machine learning models by quantifying how well they predict the target variable compared to actual values. For regression problems, we used the following regression metrics for evaluating the chosen machine learning algorithms:

- Correlation Coefficient (R^2)
- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

2.6.1 Correlation Coefficient

The coefficient of determination, or R^2, quantifies the proportion of variance in the dependent variable that can be explained by the independent variables. It ranges from 0 to 1 and reflects how well the model fits the data.

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{2.12}$$

Here,

- y_i are the actual values,
- $\hat{y}_i$ are the predicted values,
- $\bar{y}$ is the mean of the actual values y_i,
- n is the number of observations.

TSS stands for *total sum of squares*, and RSS stands for *residual sum of squares*. TSS measures the total variance of the response Y and can be thought of as the amount of variability inherent in the response before the regression is performed. In contrast, RSS measures the amount of variability that remains unexplained after the regression has been performed. Hence, R^2 measures the *proportion of variability in Y that can be explained using X*. An R^2 statistic close to 1 indicates that a significant portion of the variability in the response variable has been explained by the regression model. A number near 0 indicates that the regression

did not explain much of the variability in the response [7]. Hence, R^2 values close to 1 are generally better, as they indicate that a large proportion of the variability in the response has been explained by the regression model.

2.6.2 Mean Absolute Error

Mean Absolute Error (MAE) represents the average of the absolute differences between the predicted values and the actual values. It offers a clear measure of prediction accuracy.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.13}$$

2.6.3 Mean Squared Error

Mean Squared Error (MSE) is calculated as the average of the squared differences between predicted and actual values. It gives more weight to larger errors than Mean Absolute Error (MAE). MSE measures how closely the predicted values align with the true values for each observation. A low MSE indicates that the predicted values are close to the actual values, whereas a high MSE implies considerable discrepancies between the predicted and true values for some observations.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2.14}$$

The MSE shown in Equation 2.14 is calculated using the training data used to fit the model and should therefore be more accurately referred to as the *training MSE*. But in general, we do not care how well the method works on the training data. Instead, we focus on the accuracy of the predictions made when our method is applied to new, previously unseen test data [7].

2.6.4 Root Mean Squared Error

Root Mean Squared Error (RMSE) is the square root of the average squared differences between predicted and actual values. It provides a measure of prediction accuracy in the same units as the target variable.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2.15}$$

3 RESULTS AND DISCUSSION

In this chapter, we present the results obtained by applying the described machine learning algorithms and ensemble methods to the KC1 dataset. We begin by discussing the analysis of outliers identified in the dataset, followed by an elaboration on the performance of the applied algorithms and methods. Finally, we provide a comparative analysis of the results to offer insights into the effectiveness and efficiency of the various approaches.

3.1 ANALYSIS OF OUTLIERS

In this study, we identified 14 outliers within the dataset using the Isolation Forest algorithm. These outliers were identified as anomalies based on their feature distributions, and they exhibit distinctive characteristics that differentiate them from the majority of the data.

The outliers show extreme values or unique patterns across several features. For instance, the `PERCENT_PUB_DATA` feature shows outliers with values of either 0.0 or 100.0, suggesting that these records represent cases where the modules are either entirely public or completely private. Similarly, the `ACCESS_TO_PUB_DATA` feature is consistently 0.0 for all outliers, indicating that these records lack access to public data.

The `COUPLING_BETWEEN_OBJECTS` feature ranges from 4.0 to 24.0 among outliers, with higher values indicating stronger dependencies between software components. This may suggest that outliers involve more complex or tightly coupled code. The `DEPTH` feature, which reflects the depth of nested code, varies from 1.0 to 6.0, with some outliers having high values, pointing to more deeply nested or complex code structures.

Outliers also exhibit high values in `LACK_OF_COHESION_OF_METHODS`, reaching up to 100.0, which indicates potential cohesion issues within these modules. The `NUM_OF_CHILDREN` feature is predominantly 0.0, with some outliers having non-zero values, which could reflect varying degrees of module complexity in terms of direct interactions or dependencies.

The `FAN_IN` feature, representing the number of other modules that call or use the module, varies among outliers, suggesting differences in module usage. Additionally, the `RESPONSE_FOR_CLASS` shows a wide range, including high values such as 222.0, indicating that some outliers might be associated with more complex or time-consuming operations.

Halstead metrics, such as `sumHALSTEAD_ERROR_EST` and `sumHALSTEAD_LENGTH`, vary significantly among outliers, reflecting diverse levels of code complexity. Source code metrics like `sumLOC_TOTAL`, `sumNUM_OPERANDS`, and `sumNUM_OPERATORS` also exhibit significant variability, pointing to a mix of different code complexity profiles.

Overall, the outliers in this dataset are characterized by extreme values and complex code structures, which may impact the modeling process.

3.2 PERFORMANCE OF THE ALGORITHMS

In this section, we present the performance results for the machine learning algorithms and ensemble methods applied to the KC1 dataset, including our baseline model $\hat{y}_i = \overline{y}$, which we used for evaluating whether the machine learning models offer improvements over a simple baseline approach. This means that the baseline model serves as a reference point, and its performance is compared against that of the machine learning algorithms. If the results of the learned models are worse than those of the baseline, it would suggest that these models may not be effectively capturing the underlying patterns in the data, or that the complexity introduced by these models might be detrimental due to overfitting or other issues.

We evaluate the following algorithms: Artificial Neural Networks, Support Vector Machines, Linear Regression, Decision Trees, Random Forest, and Gradient Boosting Regressor. The evaluation is conducted using four different approaches to assess the models' sensitivity to outliers and their overall performance. Specifically, the models are trained and evaluated under the following conditions:

1. *Leave-One-Out with Outliers Included:* Each model is trained with all data points, including outliers, and tested by leaving out one data point at a time.
2. *Leave-One-Out with Outliers Excluded:* Each model is trained without the outliers and tested by leaving out one data point at a time.
3. *K-Fold Cross-Validation with Outliers Included:* The dataset is split into 6 folds, and models are trained and evaluated including the outliers.
4. *K-Fold Cross-Validation with Outliers Excluded:* The dataset is split into 6 folds, and models are trained and evaluated excluding the outliers.

The decision to use 6 folds for cross-validation was chosen arbitrarily for the size of our dataset, ensuring that each test set had at least 20 samples. This approach provided an adequate representation of the data while maintaining a robust evaluation of model performance. Through these evaluations, we aim to determine how each algorithm performs in the presence and absence of outliers and whether the improvements over the baseline model justify the added complexity of the advanced methods.

The metrics used for evaluation include the R-squared value, Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE). These metrics offer insights into the model's predictive accuracy and error distribution.

Table 3 shows the performance metrics for the baseline model alongside various algorithms using the leave-one-out cross-validation on the entire dataset. Table 4 shows the performance metrics for the baseline model alongside various algorithms using the leave-one-out cross-validation with outliers excluded.

Table 3: Model performance using leave-one-out cross-validation with outliers included

Algorithm	R^2	*MSE*	*RMSE*	*MAE*
Baseline Model	-0.0143	121.5104	11.0232	6.1313
ANN (300,)	0.3122	82.3944	9.0771	5.6043
ANN (500,)	0.2974	84.1693	9.1744	5.6167
ANN (200, 100)	0.3494	77.9313	8.8279	5.1860
ANN (300, 200, 100)	0.3471	78.2080	8.8435	4.5644
Linear Regression	0.2002	95.8104	9.7883	5.0401
SVR	0.1406	102.9445	10.1462	4.1220
Decision Tree	0.1471	102.1702	10.1079	4.6524
Random Forest	0.2678	87.7141	9.3656	4.4965
Gradient Boosting	0.1926	96.7198	9.8346	4.0508

Table 4: Model performance using leave-one-out cross-validation with outliers excluded

Algorithm	R^2	*MSE*	*RMSE*	*MAE*
Baseline Model	-0.0159	42.1828	6.4948	4.5877
ANN (300,)	0.0095	41.1281	6.4131	4.2807
ANN (500,)	0.0893	37.8147	6.1494	4.0782
ANN (200, 100)	0.1907	33.6032	5.7968	3.8078
ANN (300, 200, 100)	0.1625	34.7759	5.8971	3.4132
Linear Regression	0.2188	32.4379	5.6954	4.0303
SVR	0.0240	40.5235	6.3658	3.3095
Decision Tree	-0.3677	56.7873	7.5357	4.5470
Random Forest	0.2775	30.0001	5.4772	3.5230
Gradient Boosting	0.1254	35.3134	6.02614	3.2195

Table 5 and 6 summarize the performance metrics for the baseline model and various algorithms using 6-fold cross-validation, with and without outliers, respectively.

3.3 COMPARATIVE ANALYSIS

A comparative analysis of all models, including individual algorithms and ensemble methods, is presented in this section. The analysis includes a discussion of strengths and weaknesses among different approaches.

3.3.1 Interpretation of Results

- *Baseline Model*: The baseline model, which predicts the mean of the training data for each test instance, shows negative R^2 values because the mean is calculated separately for each training fold, leading to variations in predictions when combined across all test sets. While the model has high MSE and RMSE values, it achieves a relatively low MAE, suggesting that the average error magnitude remains fairly small, making it

Table 5: Model performance using 6-fold cross-validation with outliers included

Algorithm	R^2	*MSE*	*RMSE*	*MAE*
Baseline Model	-0.0180	121.9506	11.0431	6.1685
ANN (300,)	0.3101	82.6433	9.0908	5.7596
ANN (500,)	0.3089	82.7894	9.0989	5.7535
ANN (200, 100)	0.3516	77.6763	8.8134	5.1444
ANN (300, 200, 100)	0.2444	90.5146	9.5139	4.7660
SVR	0.0876	109.2975	10.4545	4.5002
Linear Regression	0.0898	109.0316	10.4418	5.6053
Decision Tree	-0.2996	155.6826	12.4773	5.7695
Random Forest	0.2193	93.5232	9.6707	4.8073
Gradient Boosting	0.1848	97.6498	9.8818	4.3370

Table 6: Model performance using 6-fold cross-validation with outliers excluded

Algorithm	R^2	*MSE*	*RMSE*	*MAE*
Baseline Model	-0.0573	43.8984	6.6256	4.6968
ANN (300,)	0.0415	39.7969	6.3085	4.3200
ANN (500,)	0.0747	38.4195	6.1983	4.2410
ANN (200, 100)	0.2328	31.8554	5.6441	3.7322
ANN (300, 200, 100)	0.2399	31.5587	5.6177	3.3949
SVR	-0.0147	42.1333	6.4910	3.4917
Linear Regression	0.1523	35.1981	5.9328	4.2004
Decision Tree	0.1036	37.2205	6.1009	3.5512
Random Forest	0.1439	35.5469	5.9621	3.8450
Gradient Boosting	0.0610	38.9870	6.2440	3.5889

a modest yet consistent benchmark for evaluating more complex models.

- *Artificial Neural Networks*: The ANN models demonstrate variable performance across the different cross-validation approaches. The model with a single hidden layer of 300 neurons performed better when outliers were included, showing higher R^2 values and lower errors. This trend persisted for the model with 500 neurons in a single hidden layer, indicating a reliance on outlier information to improve predictions. When using two hidden layers (200 and 100 neurons), the ANN's performance improved significantly, suggesting that deeper architectures could capture more complex patterns. However, excluding outliers still led to reduced performance, though the model remained more robust than single-layer configurations. The ANN model with three hidden layers (300, 200, and 100 neurons) showed the most balanced performance, maintaining reasonable prediction accuracy across all cross-validation approaches. It is less sensitive to the exclusion of outliers, indicating a robust model architecture.
- *Support Vector Regression*: SVR showed moderate performance with a noticeable decrease when outliers were excluded. The R^2 values were generally lower compared to ANNs, indicating difficulties in capturing the underlying data patterns. SVR's sen-

sitivity to outliers suggests a need for advanced preprocessing or parameter tuning to better handle outliers.

- *Linear Regression*: Linear Regression provided baseline performance that was outperformed by most other algorithms. It showed slight improvements when outliers were excluded but still lagged behind more complex models like ANNs and Random Forest. This indicates that Linear Regression might be too simplistic for our dataset.
- *Decision Tree Regressor*: The Decision Tree Regressor exhibited varied performance, performing poorly with high variance when outliers were included. Excluding outliers improved stability but did not significantly enhance accuracy. The model's sensitivity to outliers and tendency to overfit suggests it is less reliable for our dataset.
- *Random Forest Regressor*: Random Forest showed robust performance with reasonable R^2 values and lower errors compared to single Decision Trees. It was less affected by outliers, indicating good generalization. Random Forest's consistent performance across all approaches highlights its strength in handling complex data.
- *Gradient Boosting Regressor*: GBR exhibited promising results, especially with 6-fold cross-validation. Its performance metrics are comparable to, or slightly better than, Random Forest in some scenarios. However, it is sensitive to outliers, with improved performance when outliers are excluded. GBR's power requires careful outlier handling to achieve optimal results.

3.3.2 Comparison with Baseline

All evaluated algorithms outperformed the baseline model, which served as a simple mean predictor. ANN models and Random Forest showed the most significant improvements over the baseline, demonstrating superior capability in capturing complex patterns. Linear Regression and Decision Tree models provided modest improvements, indicating limitations in simpler models for our task.

3.3.3 Best Performing Algorithm

The best-performing algorithm was the Artificial Neural Network (ANN) with two hidden layers of 200 and 100 neurons. It consistently delivered high R^2 values and low error metrics across all cross-validation approaches, excelling even when outliers were included. Its ability to capture complex relationships made it the most effective model for predicting software defects. Two figures are provided to illustrate the performance of the model.

Figure 5 shows the predicted values versus the true values using 6-fold cross-validation. The red dots represent the predicted values, while the blue dots represent the true values. The proximity of the red dots to the blue dots indicates the accuracy of the predictions.

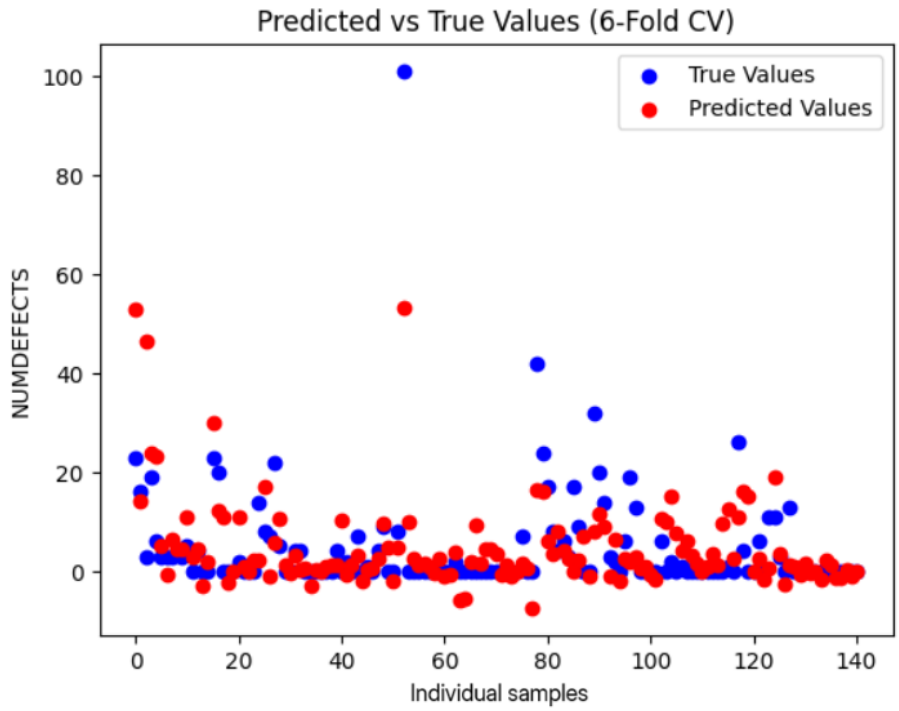

Figure 5: Predicted vs True Values (6-Fold CV) for ANN with hidden layers (200, 100).

Most of the predicted values closely match the true values, demonstrating the ANN model's effectiveness. However, some discrepancies indicate the model's limitations in handling certain instances, particularly with higher defect counts.

Figure 6 presents a histogram of the residuals for the same ANN model. The residuals are the differences between the predicted and true values. The calculation of residuals involves subtracting the true values from the predicted values for each instance.

Zero-centered residuals mean that the residuals are distributed around zero, indicating that the model does not systematically overestimate or underestimate the number of defects. This balance is a sign of an unbiased model. Most residuals falling within a narrow range around zero suggest that the model predictions are generally accurate and consistent. A histogram centered around zero with a narrow spread indicates a well-performing model, as it shows that most predictions are close to the true values.

3.3.4 Worst Performing Algorithm

The worst-performing algorithm is the Decision Tree Regressor. It shows high variance and lower R^2 values, especially with outliers included. Its sensitivity to outliers and tendency to overfit results in less reliable predictions. These findings suggest the need for ensemble methods or regularization techniques to improve its performance.

Figure 7 shows the predicted values versus the true values using 6-fold cross-validation for the model. The red dots represent the predicted values, while the blue dots represent

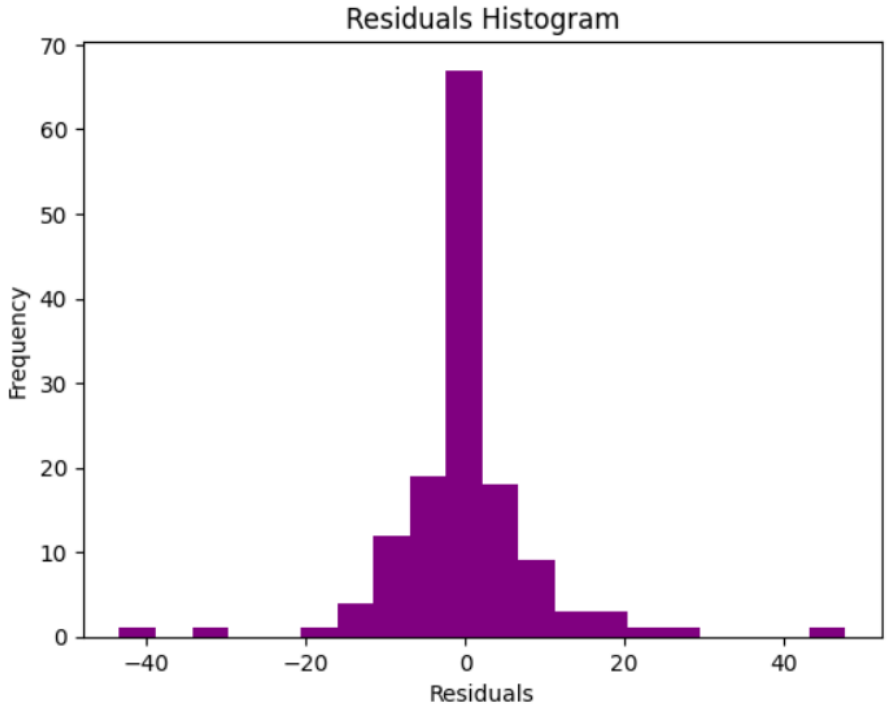

Figure 6: Residuals Histogram for ANN with hidden layers (200, 100).

the true values. The scatter of the red dots compared to the blue dots indicates significant discrepancies between the predictions and the actual values. The model struggles to accurately predict the number of defects, particularly for instances with higher defect counts, demonstrating its poor performance.

Figure 8 presents a histogram of the residuals for the Decision Tree. In this case, the residuals are not centered around zero. Instead, they are skewed to the left, with the highest frequency in the negative range. This indicates that the model tends to systematically overestimate the number of defects, as evidenced by the higher concentration of negative residuals. A left-skewed residual distribution suggests bias in the model's predictions, reflecting its limitations and the need for improvement through advanced techniques.

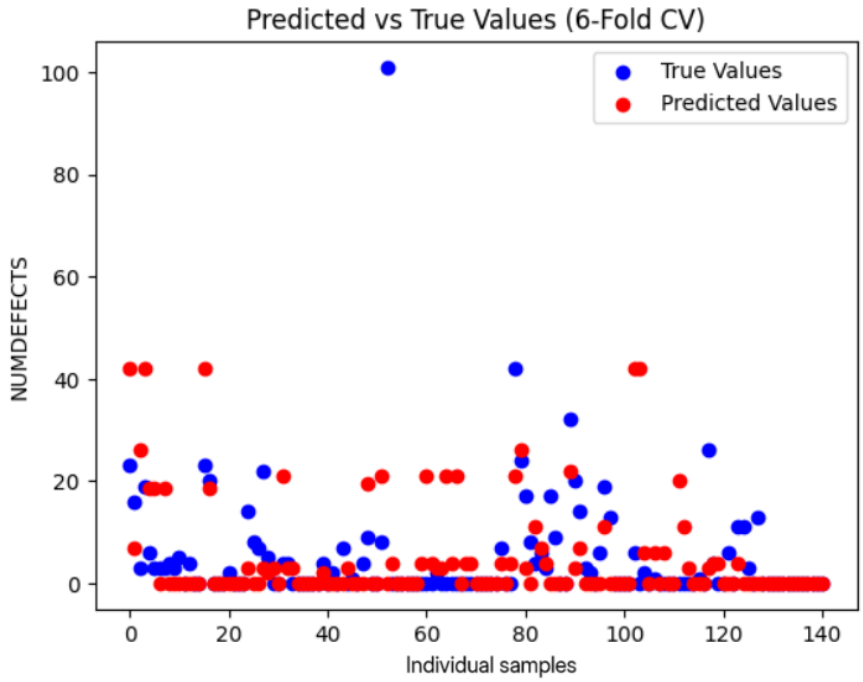

Figure 7: Predicted vs True Values (6-Fold CV) for Decision Tree Regressor.

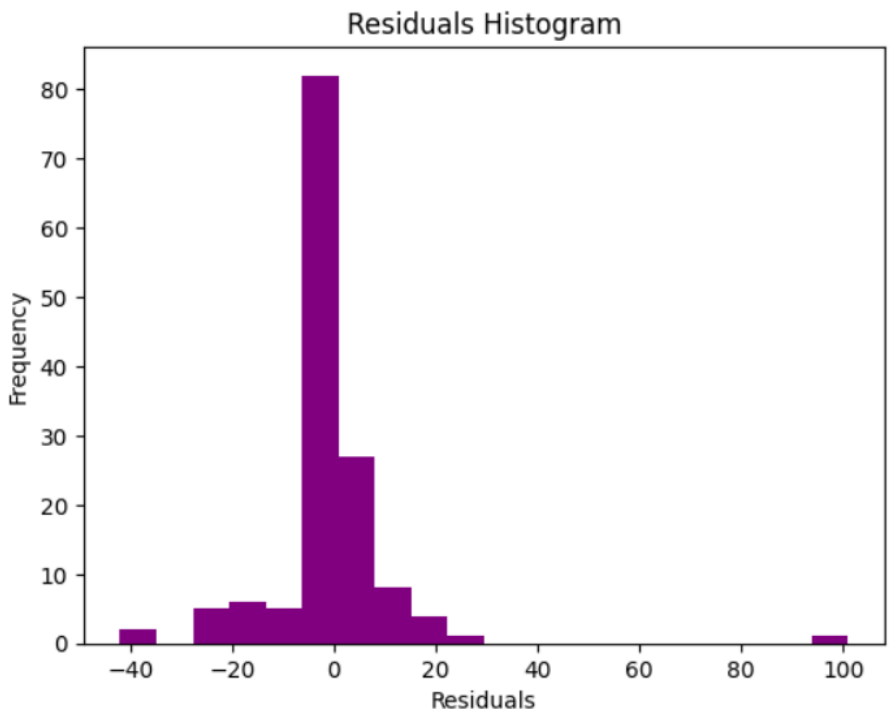

Figure 8: Residuals Histogram for Decision Tree Regressor.

4 CONCLUSION

Our primary research question explores how well can machine learning predict software defects based on features derived from McCabe and Halstead metrics. Our study has demonstrated that machine learning can marginally enhance the prediction of software defects. Among the evaluated algorithms, neural networks, have shown the best performance among the tried regression methods. Despite the increased complexity of neural networks compared to simpler algorithms, they achieved the highest predictive accuracy, as evidenced by their R^2 and RMSE metrics.

The second research question addressed the extent to which ensemble machine learning models can improve the accuracy of software defect prediction compared to traditional single-algorithm approaches. Our results indicate a notable improvement in predictive accuracy when using ensemble models. For instance, the Gradient Boosting Regressor and Random Forest models not only reduced prediction error but also improved the overall robustness of the predictions. The ensemble models' ability to combine multiple weak learners into a strong predictive model accounts for their enhanced performance. This approach mitigates the overfitting problem that is often caused by single-algorithm models, particularly in datasets with limited instances like the KC1 dataset. However, the neural network's performance surpassed that of the ensemble models, highlighting the potential of deep learning techniques in this domain.

Analysis of feature importance using Gradient Boosting Regressor identified several key factors that significantly impact the prediction of software defects. The most influential features were related to lines of code, branching, coupling between objects, Halstead metrics, and cyclomatic complexity. These metrics provide valuable insights for predictive maintenance and emphasize the relevance of both code complexity and quality measures.

Future research could build on these findings by exploring more sophisticated neural network architectures and hyperparameter tuning to further enhance predictive accuracy. Additionally, integrating other relevant metrics and domain-specific features could provide a more comprehensive understanding of defect prediction. Our study provides a solid foundation for future work in this area, highlighting the importance of advanced machine learning techniques in maintaining software quality and reliability.

In conclusion, our research validates the hypothesis that machine learning, particularly neural networks and ensemble methods, can effectively predict software defects using features derived from McCabe and Halstead metrics.

5 DALJŠI POVZETEK V SLOVENSKEM JEZIKU

Ta naloga, naslovljena *Prediktivno vzdrževanje programske opreme z uporabo strojnega učenja: Primerjalna študija algoritmov na podatkovnem naboru KC1 NASA PROMISE*, preučuje potencial tehnik strojnega učenja za izboljšanje napovedovanja napak v programski opremi z uporabo značilk, pridobljenih iz metrik McCabe in Halstead [9]. Cilj naloge je odgovoriti na ključna vprašanja o učinkovitosti strojnega učenja pri napovedovanju napak v programski opremi, relativnem izboljšanju, ki ga ponujajo ensemble modeli v primerjavi s tradicionalnimi pristopi, in ključnih dejavnikih, ki vplivajo na natančnost napovedi.

Raziskava ocenjuje, kako dobro lahko strojno učenje napoveduje napake v programski opremi z uporabo podatkovnega nabora, obogatenega z McCabe in Halstead metrikami. Te metrike kvantificirajo različne vidike kompleksnosti in kakovosti kode, kar predstavlja trdno osnovo za ocenjevanje zanesljivosti programske opreme [5]. Podatkovni nabor vsebuje 145 primerov in 95 atributov, pri čemer je `NUMDEFECT` ciljna spremenljivka, ki beleži število napak za vsako razred. Značilnosti predstavljajo numerične vrednosti, povezane s kompleksnostjo kode in strukturnimi značilnostmi, kar ponuja celovit pregled potenciala programske opreme za napake.

V tej raziskavi smo kot glavno razvojno orodje uporabili programski jezik Python, ki je podprt s številnimi knjižnicami. Knjižnico `NumPy` smo uporabili za matematične operacije, medtem ko nam je `SciPy` omogočal nalaganje in obdelavo ARFF datotečnih formatov preko svojega modula `arff`. Manipulacijo in analizo podatkov smo izvedli z uporabo knjižnice `Pandas`, ki je nudila robustna orodja za obravnavo podatkovnih okvirjev. Algoritme strojnega učenja smo implementirani z uporabo knjižnice `Scikit-Learn`, ki ponuja širok spekter modelov in metrik za ocenjevanje njihove točnosti. Za izdelavo grafov in vizualizacij smo uporabili knjižnico `Matplotlib`. Skupaj so nam ta orodja zagotavljala usklajen in učinkovit pristop k obdelavi podatkov, ocenjevanju modelov in vizualizaciji rezultatov.

Za oceno uspešnosti modelov strojnega učenja smo uporabili več algoritmov, vključno z umetnimi nevronskimi mrežami (ANN), metodo podpornih vektorjev za regresijo (SVR), linearno regresijo (LR), odločitvenimi drevesi (DT), naključnimi gozdovi (RF) in regresorjem s gradientno krepitvijo (GBR). Uspešnost modelov je bila ocenjena z uporabo metode z izločanjem enega vzorca in 6-kratnim prečnim preverjanjem, tako z vključitvijo kot izključitvijo iztopajočih primerov. Za identifikacijo izstopajočih primerov smo uporabili algoritem IsolationForest, pri čemer je bilo zaznanih 14 primerov, ki so pomembno vplivali na rezultate, zlasti pri preprostejših modelih. Med preizkušenimi modeli se je umetna nevronska mreža (ANN) z arhitekturo (200, 100) izkazala za najučinkovitejšo. Ta model je dosegel R^2 oceno 0,3516, povprečno kvadratno napako (MSE) 77,6763, koren kvadratne povprečne napake (RMSE) 8,8134 in povprečno absolutno napako (MAE) 5,1444 pri ocenjevanju z

uporabo 6-kratnega prečnega preverjanja z vključitvijo izstopajočih primerov. Uspešnost ANN poudarja njeno sposobnost zajemanja kompleksnih vzorcev in interakcij v podatkih. Nasprotno pa so bila odločitvena drevesa (Decision Trees) najmanj učinkovite, z R^2 oceno - 0,2996, MSE 155,6826, RMSE 12,4773 ter MAE 5,7695, zlasti ko so bile vključene izstopajoče vrednosti. Takšna slaba uspešnost nakazuje na občutljivost odločilnih dreves na izstopajoče vrednosti in njihovo tendenco k pretiranem prilagajanju učni množici, kar vodi do manj zanesljivih napovedi.

Študija prav tako raziskuje izboljšave, ki jih ponujajo ansambelske metode strojnega učenja v primerjavi s posameznimi algoritmi. Metode ansamblov, kot sta Random Forest in Gradient Boosting Regressor, so pokazale pomembna izboljšanja točnosti napovedi v primerjavi s preprostejšimi modeli. Random Forest je na primer dosegel R^2 oceno 0,2193, MSE 93,5232, RMSE 9,6707 in MAE 4,8073 z vključitvijo izjem, kar kaže na boljšo uspešnost v primerjavi z drugimi enostavnimi modeli. Gradient Boosting Regressor, znan po svoji sposobnosti gradnje točnih modelov z zaporednim popravljanjem napak, je prav tako dosegel konkurenčne rezultate z R^2 oceno 0,1848, MSE 97,6498, RMSE 9,8818 in MAE 4,3370. Te ugotovitve nakazujejo, da lahko ansambelske metode bistveno izboljšajo točnost napovedi napak v sistemih za predvidevanje napak v programski kodi. Poleg tega raziskava identificira več ključnih značilnosti, ki pomembno vplivajo na napovedovanje napak v podatkovnem naboru KC1. Z uporabo regresorja z gradientno okrepitvijo je študija izpostavila značilnosti programske kode, kot so `sumLOC_TOTAL`, `sumBRANCH_COUNT`, `COUPLING_BETWEEN_OBJECTS` in `maxHALSTEAD_VOLUME`. Med njimi je `sumLOC_TOTAL`, ki predstavlja skupno število vrstic kode v razredu izstopal kot še posebej pomemben, in sicer z oceno pomembnosti značilke 0.0422. To poudarja ključno vlogo metrik kompleksnosti programske kode pri napovedovanju potencialnih napak. Te ključne značilnosti zagotavljajo dragocene vpoglede v vidike kakovosti programske kode, ki so najbolj napovedni za napake, kar lahko pripomore k nadaljnjim raziskavam prediktivnega vzdrževanja programske kode.

6 BIBLIOGRAPHY

[1] Charu C. Aggarwal. *Neural Networks and Deep Learning*. Springer International Publishing, 2018.

[2] Marwa Assim, Qasem Obeidat, and Mustafa Hammad. Software defects prediction using machine learning algorithms. *International Conference on Data Analytics for Business and Industry*, 2020.

[3] Mariette Awad and Rahul Khanna. *Efficient Learning Machines: Theories, Concepts, and Applications for Engineers and System Designers*. Apress, 2015.

[4] Giuseppe Bonaccorso. *Machine Learning Algorithms*. Packt Publishing, 2017.

[5] Norman E. Fenton and Shari L. Pfleeger. *Software Metrics: A Rigorous and Practical Approach, 2nd edition*. Chapman & Hall/CRC, 1997.

[6] Jerome H. Friedman. Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4):367–378, 2002.

[7] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning, with Applications in R*. Springer, 2013.

[8] John D. Kelleher, Brian Mac Namee, and Aoife D'Arcy. *Fundamentals of Machine Learning for Predictive Data Analytics, 2nd edition*. Massachusetts Institute of Technology, 2015.

[9] A. Gunes Koru. KC1 Class-Level Numeric Defect Dataset. `http://promise.site.uottawa.ca/SERepository/datasets/kc1-class-level-numericdefect.arff`, 2005. Accessed: 2023-07-08.

[10] Gunes Koru and Hongfang Liu. An investigation of the effect of module size on defect prediction using static measures. *ACM SIGSOFT Software Engineering Notes*, 2005.

[11] Scikit learn developers. GradientBoostingRegressor. `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html#sklearn.ensemble.GradientBoostingRegressor`. Accessed: 2023-07-14.

[12] Scikit learn developers. RandomForestRegressor. `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html`. Accessed: 2023-07-12.

[13] Gilles Louppe, Louis Wehenkel, Antonio Sutera, and Pierre Geurts. Understanding variable importances in forests of randomized trees. *Advances in Neural Information Processing Systems 26*, 2013.

[14] Stephen Marshland. *Machine Learning, An Algorithmic Perspective, 2nd edition*. Chapman & Hall/CRC, 2015.

[15] Masooma Memon. ANN vs CNN vs RNN: Neural Networks Guide. `https://levity.ai/blog/neural-networks-cnn-ann-rnn`, 2022. Accessed: 2023-07-05.

[16] R. Keith Mobley. *An Introduction to Predictive Maintenance, 2nd edition*. Elsevier Science (USA), 1943.

[17] Shari Lawrence Pfleeger and Joanne M. Atlee. *Software Engineering: Theory and Practice, 4th edition*. Pearson, 1998.

[18] Scikit-learn developers. DecisionTreeRegressor. `https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html`. Accessed: 2023-07-24.

[19] Scikit-learn developers. LinearRegression. `https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html`. Accessed: 2023-07-22.

[20] Scikit-learn developers. MLPRegressor. `https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html`. Accessed: 2023-07-18.

[21] Scikit-learn developers. SVR. `https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html`. Accessed: 2023-07-20.

[22] Zhi-Hua Zhou. *Ensemble Methods, Foundations and Algorithms*. Chapman & Hall/CRC, 2012.